```
const PROXY='https://reborn-proxy.bormotyt.workers.dev';
const TARGETS={cal:2340,p:128,c:200,f:65};
const BASELINE={weight:89.1,bf:25.1,fatMass:22.4};
const GOAL_BF=20.1;
const CIRC=232.5;
const CUPS=8;
const ML_PER_CUP=250;
const MONTHS=['January','February','March','April','May','June','July','August','
const DOWS=['Su','Mo','Tu','We','Th','Fr','Sa'];
const KEY='rb5';

const SNAP_CFG=[
  {label:'Wake Up',sub:'Log sleep and recovery from this morning.',fields:[
    {id:'sleep',label:'Sleep',ph:'7:42',step:'',inputType:'sleep'},
    {id:'recovery',label:'Recovery %',ph:'e.g. 72',step:'1'},
  ]},
  {label:'1 PM',sub:'Log your mid-day Whoop stats.',fields:[
    {id:'strain',label:'Strain so far',ph:'e.g. 8.2',step:'0.1'},
    {id:'burned',label:'Cals Burned',ph:'e.g. 1200',step:'10'},
    {id:'steps',label:'Steps',ph:'e.g. 4500',step:'100'},
  ]},
  {label:'10 PM',sub:'Log your final Whoop stats for the day.',fields:[
    {id:'strain',label:'Final Strain',ph:'e.g. 14.5',step:'0.1'},
    {id:'burned',label:'Total Burned',ph:'e.g. 2800',step:'10'},
    {id:'steps',label:'Total Steps',ph:'e.g. 9200',step:'100'},
  ]},
];

const todayKey=()=>new Date().toISOString().slice(0,10);
const load=(k,d)=>{try{return JSON.parse(localStorage.getItem(k))??d}catch{return
const save=(k,v)=>localStorage.setItem(k,JSON.stringify(v));
const gv=id=>document.getElementById(id);

const DEFAULT_QA=[
  {id:'ghost-pre',  name:'GHOST Pre',      emoji:'⚡',calories:5,  protein:0, car
  {id:'ghost-whey', name:'GHOST Whey',     emoji:'🥛',calories:110,protein:25,car
  {id:'banana',     name:'Banana',         emoji:'🍌',calories:89, protein:1, car
  {id:'date',       name:'Date',           emoji:'🟤',calories:20, protein:0, car
  {id:'egg',        name:'Hard Boiled Egg',emoji:'🥚',calories:78, protein:6, car
  {id:'nada',       name:'NADA Yogurt',    emoji:'🫙',calories:130,protein:30,car
];

// STATE
```

```
let meals      =load(`${KEY}_meals_${todayKey()}`,[]);
let whoopSnaps =load(`${KEY}_whoopsnaps_${todayKey()}`,[null,null,null]);
let cups       =parseInt(localStorage.getItem(`${KEY}_cups_${todayKey()}`)||'0');
let entries    =load(`${KEY}_entries`,[]);
let quickItems =load(`${KEY}_quickitems`,DEFAULT_QA);
let calViewDate=new Date();
let calSelKey  =todayKey();
let mealB64    =null;
let ingredients=[];
let activeTab  =0;

// BOOT
const _n=new Date();
gv('hdr-date').innerHTML=_n.toLocaleDateString('en-US',{weekday:'short',month:'sh
gv('coach-date').textContent=_n.toLocaleDateString('en-US',{weekday:'long',month:
renderAll();
initWelcomeCard();

function renderAll(){
  renderWhoopCard();renderSummary();renderRings();
  renderCups();renderFoodList();renderQuickAdd();
  renderProgress();buildCalendar();updateCoachStats();
  // refresh smart subtitle with latest stats
  const sub=gv('wc-sub');if(sub)sub.textContent=getSmartSub();
}

// NAV
const PAGE_ORDER=['today','coach','weekly','calendar','progress'];
let _currentPage='today';
function showPage(id,btn){
  const pages=document.querySelectorAll('.page');
  const fromIdx=PAGE_ORDER.indexOf(_currentPage);
  const toIdx=PAGE_ORDER.indexOf(id);
  const dir=toIdx>=fromIdx?1:-1; // 1=going right, -1=going left
  const outgoing=gv('pg-'+_currentPage);
  const incoming=gv('pg-'+id);
  if(outgoing&&outgoing!==incoming){
    outgoing.classList.add(dir===1?'page-exit-left':'page-exit-right');
    outgoing.addEventListener('animationend',()=>{
      outgoing.classList.remove('active','page-exit-left','page-exit-right');
    },{once:true});
  }
  document.querySelectorAll('.nb').forEach(b=>b.classList.remove('active'));
  incoming.classList.remove('page-exit-left','page-exit-right');
  incoming.classList.add('active',dir===1?'page-enter-right':'page-enter-left');
  incoming.addEventListener('animationend',()=>{
    incoming.classList.remove('page-enter-right','page-enter-left');
```

```
  },{once:true});
  btn.classList.add('active');
  incoming.scrollTop=0;
  _currentPage=id;
  if(id==='calendar')buildCalendar();
  if(id==='coach')updateCoachStats();
}

// ── WHOOP 3-SNAPSHOT ──
function selectWhoopTab(i){
  activeTab=i;
  document.querySelectorAll('.wh-tab').forEach((t,j)=>t.classList.toggle('active'
  renderWhoopSnap();
}
function renderWhoopCard(){
  whoopSnaps.forEach((s,i)=>{const t=gv(`wh-tab-${i}`);if(t)t.classList.toggle('h
  renderWhoopSnap();
}
function renderWhoopSnap(){
  const snap=whoopSnaps[activeTab];
  const el=gv('wh-snap-display');
  if(!snap){el.innerHTML=`<div class="wh-empty">No data yet. Tap <strong>+ Log Sn
  const cfg=SNAP_CFG[activeTab];
  const items=cfg.fields.map(f=>{
    const v=snap[f.id];if(v==null)return'';
    let d;
    if(f.id==='steps')d=Number(v).toLocaleString();
    else if(f.id==='recovery')d=v+'%';
    else if(f.id==='sleep'){const hh=Math.floor(v),mm=Math.round((v-hh)*60);d=hh+
    else d=v;
    return`<div class="wh-s"><div class="wh-v">${d}</div><div class="wh-l">${f.la
  }).filter(Boolean);
  const cols=items.length<=2?'wh-grid-2':'wh-grid-3';
  el.innerHTML=`<div class="wh-grid ${cols}">${items.join('')}</div>`;
}
function openWhoopModal(){
  const cfg=SNAP_CFG[activeTab];
  const snap=whoopSnaps[activeTab]||{};
  gv('whoop-modal-title').textContent=`WHOOP · ${cfg.label}`;
  gv('whoop-modal-sub').textContent=cfg.sub;
  let html='';
  for(let i=0;i<cfg.fields.length;i+=2){
    html+='<div class="frow">';
    for(let j=i;j<Math.min(i+2,cfg.fields.length);j++){
      const f=cfg.fields[j];
      if(f.inputType==='sleep'){
        const dec=snap[f.id]; const hh=dec!=null?Math.floor(dec):''; const mm=dec
```

```
        const val=dec!=null?`${hh}:${mm}`:'';
        html+=`<div><label class="flbl">${f.label} <span style="color:var(--muted
      } else {
        html+=`<div><label class="flbl">${f.label}</label><input type="number" id
      }
    }
    html+='</div>';
  }
  if(activeTab===0||activeTab===2){
    const g=snap.goal||'cut';
    html+=`<div class="frow"><div><label class="flbl">Goal</label><select id="wf-
  }
  gv('whoop-modal-fields').innerHTML=html;
  gv('whoop-modal').classList.add('open');
}
function closeWhoopModal(){gv('whoop-modal').classList.remove('open');}
function saveWhoop(){
  const cfg=SNAP_CFG[activeTab];
  const snap={};
  cfg.fields.forEach(f=>{
    const el=gv(`wf-${f.id}`);if(!el||el.value==='')return;
    if(f.inputType==='sleep'){
      const v=el.value.trim();
      snap[f.id]=v.includes(':')?Math.round((parseInt(v.split(':')[0]||0)+parseIn
    } else { snap[f.id]=f.id==='steps'?parseInt(el.value):parseFloat(el.value); }
  });
  const ge=gv('wf-goal');if(ge)snap.goal=ge.value;
  whoopSnaps[activeTab]=snap;
  save(`${KEY}_whoopsnaps_${todayKey()}`,whoopSnaps);
  closeWhoopModal();renderAll();
}
function getCalTarget(){
  // Food intake target is fixed at TARGETS.cal (2,340 kcal cut target).
  // WHOOP burned = total daily expenditure — it's informational only.
  // The deficit is already baked into TARGETS.cal; we don't adjust food
  // intake up or down based on how much was burned.
  const g=(whoopSnaps[0]&&whoopSnaps[0].goal)||(whoopSnaps[2]&&whoopSnaps[2].goal
  if(g==='bulk')  return TARGETS.cal+500;
  if(g==='maintain') return TARGETS.cal+200;
  return TARGETS.cal; // cut — default
}

// SUMMARY
function getTotals(arr){return(arr||meals).reduce((a,m)=>({cal:a.cal+m.calories,p

// Animated number counter — cancels previous if called again on same element
function countUp(el,toVal,duration,suffix,prefix){
```

```
  if(!el)return;
  const prevTo=el._countTo;
  el._countTo=toVal;
  if(prevTo===toVal)return; // no change, skip
  const startVal=parseFloat(el._countFrom||0);
  el._countFrom=toVal;
  const startTime=performance.now();
  const isInt=Number.isInteger(toVal);
  function step(now){
    if(el._countTo!==toVal)return; // superseded
    const p=Math.min((now-startTime)/duration,1);
    // ease out cubic
    const ease=1-Math.pow(1-p,3);
    const cur=startVal+(toVal-startVal)*ease;
    el.textContent=(prefix||'')+(isInt?Math.round(cur):(Math.round(cur*10)/10))+(
    if(p<1)requestAnimationFrame(step);
  }
  requestAnimationFrame(step);
}

function renderSummary(){
  const t=getTotals();
  const tgt=getCalTarget(),rem=Math.round(tgt-t.cal),pct=t.cal/tgt;
  // Animate calorie number
  const calEl=gv('cal-num');
  const prevCal=calEl._countTo||0;
  const targetCal=Math.round(t.cal);
  // Use innerHTML for the kcal sup — animate a child span
  if(!calEl.querySelector('.cal-count-n')){
    calEl.innerHTML='<span class="cal-count-n">0</span><sup>kcal</sup>';
  }
  const numSpan=calEl.querySelector('.cal-count-n');
  numSpan._countTo=numSpan._countTo||0;
  const oldTarget=numSpan._countTo;
  numSpan._countTo=targetCal;
  if(oldTarget!==targetCal){
    // pulse the hero number
    calEl.classList.remove('cal-pulse');
    void calEl.offsetWidth;
    calEl.classList.add('cal-pulse');
    const startV=parseFloat(numSpan._countFrom||0);
    numSpan._countFrom=targetCal;
    const dur=600,start=performance.now();
    (function step(now){
      if(numSpan._countTo!==targetCal)return;
      const p=Math.min((now-start)/dur,1);
      const ease=1-Math.pow(1-p,3);
```

```
      numSpan.textContent=Math.round(startV+(targetCal-startV)*ease);
      if(p<1)requestAnimationFrame(step);
    })(performance.now());
  }
  const rEl=gv('cal-rem');
  rEl.textContent=rem>=0?rem+' left':Math.abs(rem)+' over';
  rEl.className='cal-rem'+(rem<0?' over':pct>0.85?' good':'');
  gv('cal-tlbl').textContent='of '+tgt.toLocaleString()+' kcal target';
  const bar=gv('cal-bar');
  bar.style.width=Math.min(pct*100,100)+'%';
  bar.className='pbar-f'+(t.cal>tgt?' over':pct>0.85?' warn':'');
}

// RINGS
function renderRings(){
  const t=getTotals();
  setRing('rp','rp-v','rp-pct',t.p,TARGETS.p,'g',0);
  setRing('rc','rc-v','rc-pct',t.c,TARGETS.c,'g',80);
  setRing('rf','rf-v','rf-pct',t.f,TARGETS.f,'g',160);
}
function setRing(id,vid,pid,cur,tgt,unit,delay){
  const raw=cur/tgt;
  const targetOffset=CIRC-Math.min(raw,1)*CIRC;
  const over=raw>1;
  const strokeEl=gv(id);
  const valEl=gv(vid);
  const pctEl=gv(pid);
  // ring colour
  strokeEl.style.stroke=over?'var(--red)':'';
  pctEl.style.color=over?'var(--red)':'';
  // animate ring draw
  const prevOffset=parseFloat(strokeEl._prevOffset!=null?strokeEl._prevOffset:CIR
  if(Math.abs(prevOffset-targetOffset)>0.5){
    strokeEl._prevOffset=targetOffset;
    // briefly reset to start then let CSS transition carry it
    strokeEl.style.transition='none';
    strokeEl.style.strokeDashoffset=prevOffset;
    setTimeout(()=>{
      strokeEl.style.transition='stroke-dashoffset .9s cubic-bezier(.34,1.56,.64,
      strokeEl.style.strokeDashoffset=targetOffset;
    }, delay||0);
  }
  // count up value label
  const prevVal=parseFloat(valEl._countFrom||0);
  valEl._countFrom=cur;
  const dur=700,start=performance.now()+delay;
  (function stepV(now){
```

```
    if(now<start){requestAnimationFrame(stepV);return;}
    const p=Math.min((now-start)/dur,1);
    const ease=1-Math.pow(1-p,3);
    valEl.textContent=Math.round(prevVal+(cur-prevVal)*ease)+unit;
    if(p<1)requestAnimationFrame(stepV);
  })(performance.now());
  // count up pct label
  const prevPct=parseFloat(pctEl._countFrom||0);
  pctEl._countFrom=raw*100;
  (function stepP(now){
    if(now<start){requestAnimationFrame(stepP);return;}
    const p=Math.min((now-start)/dur,1);
    const ease=1-Math.pow(1-p,3);
    pctEl.textContent=Math.round(prevPct+(raw*100-prevPct)*ease)+'%';
    if(p<1)requestAnimationFrame(stepP);
  })(performance.now());
}

// CUPS
function renderCups(){
  const grid=gv('cups-grid');grid.innerHTML='';
  for(let i=0;i<CUPS;i++){
    const btn=document.createElement('button');
    btn.className='cup-btn'+(i<cups?' filled':'');
    btn.innerHTML=i<cups?'💧':'○';
    btn.addEventListener('click',()=>{cups=i<cups?i:i+1;localStorage.setItem(`${K
    grid.appendChild(btn);
  }
  gv('water-cups').textContent=cups;
  gv('water-ml-display').textContent=(cups*ML_PER_CUP).toLocaleString();
  gv('water-bar').style.width=Math.min((cups/CUPS)*100,100)+'%';
}
function resetWater(){cups=0;localStorage.setItem(`${KEY}_cups_${todayKey()}`,0);

// QUICK ADD
function renderQuickAdd(){
  const el=gv('qa-scroll');
  el.innerHTML='';
  for(const item of quickItems){
    const div=document.createElement('div');
    div.className='qa-item';
    div.innerHTML=`<div class="qa-icon">${item.emoji}</div><div class="qa-name">$
    div.querySelector('.qa-del').addEventListener('click',e=>{e.stopPropagation()
    div.addEventListener('click',()=>quickLog(item.id));
    el.appendChild(div);
  }
  const addBtn=document.createElement('div');
```

```
  addBtn.className='qa-add';
  addBtn.innerHTML=`<div style="font-size:20px">+</div><div style="font-size:9px'
  addBtn.addEventListener('click',openQAModal);
  el.appendChild(addBtn);
}
function deleteQAItem(id){
  if(!confirm('Remove from Quick Add?'))return;
  quickItems=quickItems.filter(i=>i.id!==id);
  save(`${KEY}_quickitems`,quickItems);
  renderQuickAdd();
}
let _quickLock=false;
function quickLog(id){
  if(_quickLock)return;
  const item=quickItems.find(i=>i.id===id);if(!item)return;
  _quickLock=true;
  meals.push({name:item.name,emoji:item.emoji,calories:item.calories,protein:item
  save(`${KEY}_meals_${todayKey()}`,meals);
  renderAll();
  setTimeout(()=>{_quickLock=false;},600);
}
function openQAModal(){['qa-name-in','qa-emoji-in','qa-cal-in','qa-p-in','qa-c-in
function closeQAModal(){gv('qa-modal').classList.remove('open');}
async function aiLookupQA(){
  const name=gv('qa-name-in').value.trim();if(!name){alert('Enter a name first.')
  gv('qa-loading').classList.add('show');
  try{
    const res=await fetch(PROXY,{method:'POST',headers:{'Content-Type':'applicati
      body:JSON.stringify({model:'claude-sonnet-4-20250514',max_tokens:300,
        system:'Return ONLY valid JSON, no markdown: {"emoji":"single emoji","cal
        messages:[{role:'user',content:`Nutrition facts for: ${name}. Use officia
    const data=await res.json();
    const p=JSON.parse(data.content.map(b=>b.text||'').join('').replace(/```json|
    if(p.emoji)gv('qa-emoji-in').value=p.emoji;
    if(p.calories)gv('qa-cal-in').value=p.calories;
    if(p.protein)gv('qa-p-in').value=p.protein;
    if(p.carbs)gv('qa-c-in').value=p.carbs;
    if(p.fat)gv('qa-f-in').value=p.fat;
  }catch(e){alert('Lookup failed.');}
  finally{gv('qa-loading').classList.remove('show');}
}
function saveQAItem(){
  const name=gv('qa-name-in').value.trim();if(!name){alert('Enter a name.');retur
  quickItems.push({id:'custom_'+Date.now(),name,emoji:gv('qa-emoji-in').value||''
    calories:parseFloat(gv('qa-cal-in').value)||0,protein:parseFloat(gv('qa-p-in'
    carbs:parseFloat(gv('qa-c-in').value)||0,fat:parseFloat(gv('qa-f-in').value)|
  save(`${KEY}_quickitems`,quickItems);closeQAModal();renderQuickAdd();
```

```
}

// FOOD LIST
function renderFoodList(){
  const el=gv('food-list');
  if(!meals.length){el.innerHTML='<div class="empty-st"><span class="empty-icon">
  el.innerHTML='';
  for(let i=0;i<meals.length;i++){
    const m=meals[i];const div=document.createElement('div');div.className='fi';
    const th=m.thumb?`<img class="fi-thumb" src="${m.thumb}" style="width:46px;he
    const pPct=Math.min(Math.round(m.protein/TARGETS.p*100),100);
    const cPct=Math.min(Math.round(m.carbs/TARGETS.c*100),100);
    const fPct=Math.min(Math.round(m.fat/TARGETS.f*100),100);
    div.innerHTML=`<div class="fi-top">${th}<div class="fi-info"><div class="fi-n
    div.querySelector('.fi-del').addEventListener('click',(e)=>{e.stopPropagation
    div.addEventListener('click',()=>openMealDetail(i));
    el.appendChild(div);
  }
}
function deleteMeal(i){meals.splice(i,1);save(`${KEY}_meals_${todayKey()}`,meals)

// MEAL DETAIL MODAL
function openMealDetail(idx){
  const m=meals[idx];if(!m)return;
  const modal=gv('meal-detail-modal');if(!modal)return;
  // reset edit + analysis state
  const ep=gv('mdd-edit-panel');if(ep)ep.style.display='none';
  const eb=gv('mdd-edit-btn');if(eb){eb.style.display='';eb.disabled=false;}
  gv('mdd-analysis-wrap').style.display='none';
  gv('mdd-analysis-body').textContent='';
  gv('mdd-analyze-btn').style.display='block';
  gv('mdd-analyze-btn').disabled=false;
  gv('mdd-analyze-btn').textContent='✦ Analyse with Coach';
  // header
  gv('mdd-emoji').textContent=m.emoji||'🍽';
  gv('mdd-name').textContent=m.name;
  const ts=m.loggedAt?new Date(m.loggedAt).toLocaleTimeString('en-US',{hour:'2-di
  gv('mdd-time').textContent=ts?`Logged at ${ts}`:'';
  // macro summary chips
  gv('mdd-cal').textContent=Math.round(m.calories)+' kcal';
  gv('mdd-p').textContent=Math.round(m.protein*10)/10+'g P';
  gv('mdd-c').textContent=Math.round(m.carbs*10)/10+'g C';
  gv('mdd-f').textContent=Math.round(m.fat*10)/10+'g F';
  // ingredient breakdown
  const ingEl=gv('mdd-ingredients');
  if(m.ingredients&&m.ingredients.length>1){
    ingEl.innerHTML=m.ingredients.map(ing=>`
```

```
      <div class="mdd-ing-row">
        <span class="mdd-ing-emoji">${ing.emoji||'•'}</span>
        <div class="mdd-ing-info">
          <div class="mdd-ing-name">${ing.name}</div>
          <div class="mdd-ing-macros">
            <span style="color:var(--blue2)">${ing.calories} kcal</span>
            <span style="color:var(--pc)">${ing.protein}g P</span>
            <span style="color:var(--cc)">${ing.carbs}g C</span>
            <span style="color:var(--fc)">${ing.fat}g F</span>
          </div>
        </div>
      </div>`).join('');
    gv('mdd-ing-section').style.display='block';
  } else {
    gv('mdd-ing-section').style.display='none';
  }
  // progress bars showing this meal's contribution to daily targets
  const pPct=Math.min(Math.round(m.protein/TARGETS.p*100),100);
  const cPct=Math.min(Math.round(m.carbs/TARGETS.c*100),100);
  const fPct=Math.min(Math.round(m.fat/TARGETS.f*100),100);
  const calPct=Math.min(Math.round(m.calories/getCalTarget()*100),100);
  gv('mdd-bar-cal').style.width=calPct+'%';
  gv('mdd-bar-p').style.width=pPct+'%';
  gv('mdd-bar-c').style.width=cPct+'%';
  gv('mdd-bar-f').style.width=fPct+'%';
  gv('mdd-pct-cal').textContent=calPct+'%';
  gv('mdd-pct-p').textContent=pPct+'%';
  gv('mdd-pct-c').textContent=cPct+'%';
  gv('mdd-pct-f').textContent=fPct+'%';
  modal.classList.add('open');
  modal._mealIdx=idx;
}
function closeMealDetail(){
  const modal=gv('meal-detail-modal');
  if(modal)modal.classList.remove('open');
}
function openMealEdit(){
  const modal=gv('meal-detail-modal');
  const idx=modal._mealIdx;
  const m=meals[idx];if(!m)return;
  gv('mdd-in-cal').value=Math.round(m.calories);
  gv('mdd-in-p').value=Math.round(m.protein*10)/10;
  gv('mdd-in-c').value=Math.round(m.carbs*10)/10;
  gv('mdd-in-f').value=Math.round(m.fat*10)/10;
  gv('mdd-edit-panel').style.display='block';
  gv('mdd-edit-btn').style.display='none';
  gv('mdd-in-cal').focus();
```

```
}
function closeMealEdit(){
  gv('mdd-edit-panel').style.display='none';
  gv('mdd-edit-btn').style.display='';
}
function saveMealEdit(){
  const modal=gv('meal-detail-modal');
  const idx=modal._mealIdx;
  const m=meals[idx];if(!m)return;
  const n=(id,fb)=>{const v=parseFloat(gv(id).value);return isNaN(v)?fb:v;};
  m.calories=Math.round(n('mdd-in-cal',m.calories));
  m.protein =Math.round(n('mdd-in-p', m.protein)*10)/10;
  m.carbs   =Math.round(n('mdd-in-c', m.carbs)  *10)/10;
  m.fat     =Math.round(n('mdd-in-f', m.fat)    *10)/10;
  save(`${KEY}_meals_${todayKey()}`,meals);
  renderAll();
  // refresh chips and bars in the modal
  gv('mdd-cal').textContent=m.calories+' kcal';
  gv('mdd-p').textContent=m.protein+'g P';
  gv('mdd-c').textContent=m.carbs+'g C';
  gv('mdd-f').textContent=m.fat+'g F';
  const calPct=Math.min(Math.round(m.calories/getCalTarget()*100),100);
  const pPct=Math.min(Math.round(m.protein/TARGETS.p*100),100);
  const cPct=Math.min(Math.round(m.carbs/TARGETS.c*100),100);
  const fPct=Math.min(Math.round(m.fat/TARGETS.f*100),100);
  gv('mdd-bar-cal').style.width=calPct+'%';gv('mdd-pct-cal').textContent=calPct+'
  gv('mdd-bar-p').style.width=pPct+'%';gv('mdd-pct-p').textContent=pPct+'%';
  gv('mdd-bar-c').style.width=cPct+'%';gv('mdd-pct-c').textContent=cPct+'%';
  gv('mdd-bar-f').style.width=fPct+'%';gv('mdd-pct-f').textContent=fPct+'%';
  closeMealEdit();
}

async function analyzeMealDetail(){
  const modal=gv('meal-detail-modal');
  const idx=modal._mealIdx;
  const m=meals[idx];if(!m)return;
  const btn=gv('mdd-analyze-btn');
  btn.disabled=true;btn.textContent='Analysing…';
  gv('mdd-analysis-wrap').style.display='block';
  gv('mdd-analysis-body').textContent='';
  gv('mdd-analysis-loading').style.display='block';
  const t=getTotals();
  const tgt=getCalTarget();
  const remaining={cal:Math.round(tgt-t.cal),p:Math.round(TARGETS.p-t.p),c:Math.r
  const prompt=`You are a concise fitness coach. Borna is 26M, 89.1kg, 25.1% BF,

He just ate: ${m.name} — ${Math.round(m.calories)} kcal, ${m.protein}g P, ${m.car
```

```
${m.ingredients&&m.ingredients.length>1?'Ingredients: '+m.ingredients.map(i=>`${i

After this meal, remaining for the day: ${remaining.cal} kcal, ${remaining.p}g P,

Give a 2-3 sentence honest assessment: how well this meal fits his cut goals, wha
  try{
    const res=await fetch(PROXY,{method:'POST',headers:{'Content-Type':'applicati
      body:JSON.stringify({model:'claude-sonnet-4-20250514',max_tokens:200,
        messages:[{role:'user',content:prompt}]})});
    const data=await res.json();
    const text=data.content.map(b=>b.text||'').join('').trim();
    gv('mdd-analysis-loading').style.display='none';
    gv('mdd-analysis-body').textContent=text;
    btn.style.display='none';
  }catch(err){
    gv('mdd-analysis-loading').style.display='none';
    gv('mdd-analysis-body').textContent='Analysis failed. Try again.';
    btn.disabled=false;btn.textContent='✦ Analyse with Coach';
  }
}

// LOG MODAL
function openLogModal(){resetLogModal();gv('log-modal').classList.add('open');}
function closeLogModal(){gv('log-modal').classList.remove('open');resetLogModal()
function resetLogModal(){
  mealB64=null;ingredients=[];
  gv('preview').style.display='none';gv('ph-ph').style.display='block';
  gv('photo-zone').classList.remove('has-img');
  gv('meal-desc').value='';gv('ing-results').style.display='none';const li=gv('lo
  gv('ing-list').innerHTML='';gv('btn-analyze').textContent='ANALYSE MEAL';
  gv('btn-analyze').disabled=false;gv('meal-loading').classList.remove('show');
  gv('file-meal').value='';
  const aif=gv('add-ing-form');if(aif)aif.style.display='none';
}
function handleMealPhoto(e){
  const file=e.target.files[0];if(!file)return;
  const r=new FileReader();
  r.onload=ev=>{mealB64=ev.target.result.split(',')[1];gv('preview').src=ev.targe
  r.readAsDataURL(file);
}

async function analyzeMeal(){
  const desc=gv('meal-desc').value.trim();
  if(!mealB64&&!desc){alert('Add a photo or description first.');return;}
  gv('btn-analyze').disabled=true;gv('meal-loading').classList.add('show');gv('in
  const content=[];
  if(mealB64)content.push({type:'image',source:{type:'base64',media_type:'image/j
```

```
  const prompt=desc?(mealB64?`Analyse this food. Context: "${desc}". Identify eve
  content.push({type:'text',text:prompt});
  try{
    const res=await fetch(PROXY,{method:'POST',headers:{'Content-Type':'applicati
      body:JSON.stringify({model:'claude-sonnet-4-20250514',max_tokens:1500,
        system:`Precise nutrition expert. Identify each ingredient separately.
Return ONLY valid JSON, no markdown:
{"confidence":"high"|"medium"|"low","confidence_tip":"one sentence or empty","ing
Identify 2-8 ingredients. Include fibre, sugar, sodium where known (use 0 if unkn
        messages:[{role:'user',content}]})});
    const data=await res.json();
    const raw=data.content.map(b=>b.text||'').join('').replace(/```json|```/g,'')
    const parsed=JSON.parse(raw);
    ingredients=parsed.ingredients.map((ing,i)=>({...ing,id:i,portion_multiplier:
    const conf=parsed.confidence||'medium';
    const cm={high:{cls:'conf-high',icon:'✓',label:'High Confidence'},medium:{cls
    const c=cm[conf];
    gv('conf-badge-wrap').innerHTML=`<div class="conf-badge ${c.cls}">${c.icon} $
    const te=gv('conf-tip');
    if(parsed.confidence_tip&&conf!=='high'){te.textContent='💡 '+parsed.confiden
    else te.classList.remove('show');
    renderIngredients();gv('ing-results').style.display='block';
    gv('btn-analyze').textContent='RE-ANALYSE';
  }catch(err){alert('Analysis failed. Please try again.');console.error(err);}
  finally{gv('btn-analyze').disabled=false;gv('meal-loading').classList.remove('s
}

function renderIngredients(){
  const el=gv('ing-list');el.innerHTML='';
  for(const ing of ingredients){
    const div=document.createElement('div');
    const pm=ing.portion_multiplier;
    const cal=Math.round(ing.calories*pm),p=Math.round(ing.protein*pm*10)/10,c=Ma
    if(ing._editing){
      div.className='ing-item'+(ing.selected?' selected':'')+' ing-editing';
      div.innerHTML=`
        <div class="ing-top">
          <div class="ing-check" onclick="toggleIng(${ing.id})"><span class="ing-
          <div class="ing-emoji">${ing.emoji}</div>
          <div class="ing-name" style="font-size:13px">${ing.name}</div>
          <span class="ing-edit-hint">editing</span>
        </div>
        <div class="ing-edit-grid">
          <div class="ing-edit-col"><label class="ing-edit-lbl">kcal</label><inpu
          <div class="ing-edit-col"><label class="ing-edit-lbl">Protein g</label>
          <div class="ing-edit-col"><label class="ing-edit-lbl">Carbs g</label><i
          <div class="ing-edit-col"><label class="ing-edit-lbl">Fat g</label><inp
```

```
        </div>
        <div class="ing-edit-actions">
          <button class="ing-save-btn" onclick="saveIngEdit(${ing.id})">✓ Save</b
          <button class="ing-cancel-btn" onclick="cancelIngEdit(${ing.id})">Cance
        </div>`;
    } else {
      div.className='ing-item'+(ing.selected?' selected':'');
      const pBtns=[.5,.75,1,1.5,2].map(v=>`<button class="pbtn${pm===v?' active':
      div.innerHTML=`
        <div class="ing-top">
          <div class="ing-check" onclick="toggleIng(${ing.id})"><span class="ing-
          <div class="ing-emoji">${ing.emoji}</div>
          <div class="ing-name">${ing.name}</div>
          <button class="ing-pencil" onclick="startIngEdit(${ing.id})">✏️</button
        </div>
        <div class="ing-macros"><span class="ing-m" style="color:var(--muted)">${
        <div class="portion-row"><span class="portion-lbl">Portion:</span><div cl
    }
    el.appendChild(div);
  }
  updateSelTotals();
}
function toggleIng(id){const ing=ingredients.find(i=>i.id===id);if(ing){ing.selec
function setMultiplier(id,val){const ing=ingredients.find(i=>i.id===id);if(ing){i
function startIngEdit(id){
  ingredients.forEach(i=>{if(i.id!==id)delete i._editing;});
  const ing=ingredients.find(i=>i.id===id);
  if(ing){ing._editing=true;renderIngredients();}
}
function cancelIngEdit(id){
  const ing=ingredients.find(i=>i.id===id);
  if(ing){delete ing._editing;renderIngredients();}
}
function saveIngEdit(id){
  const ing=ingredients.find(i=>i.id===id);if(!ing)return;
  const n=(elId,fb)=>{const v=parseFloat(document.getElementById(elId)?.value);re
  ing.calories=n(`ied-cal-${id}`,ing.calories);
  ing.protein =n(`ied-p-${id}`, ing.protein);
  ing.carbs   =n(`ied-c-${id}`, ing.carbs);
  ing.fat     =n(`ied-f-${id}`, ing.fat);
  ing.portion_multiplier=1;
  ing.portion='custom';
  delete ing._editing;
  renderIngredients();
}
function updateSelTotals(){
  const sel=ingredients.filter(i=>i.selected);
```

```
  const t=sel.reduce((a,i)=>({cal:a.cal+i.calories*i.portion_multiplier,p:a.p+i.p
  gv('sel-cal').textContent=Math.round(t.cal)+' kcal';
  gv('sel-p').textContent=Math.round(t.p)+'g P';
  gv('sel-c').textContent=Math.round(t.c)+'g C';
  gv('sel-f').textContent=Math.round(t.f)+'g F';
  renderLogImpact();
}

// TOTAL MACRO OVERRIDE
function openTotalEdit(){
  // pre-fill with current computed totals
  const sel=ingredients.filter(i=>i.selected);
  const t=sel.reduce((a,i)=>({cal:a.cal+i.calories*i.portion_multiplier,p:a.p+i.p
  gv('tot-cal').value=Math.round(t.cal);
  gv('tot-p').value=Math.round(t.p*10)/10;
  gv('tot-c').value=Math.round(t.c*10)/10;
  gv('tot-f').value=Math.round(t.f*10)/10;
  gv('sel-totals-view').style.display='none';
  gv('sel-totals-edit').style.display='block';
  gv('tot-cal').focus();
}
function closeTotalEdit(){
  gv('sel-totals-edit').style.display='none';
  gv('sel-totals-view').style.display='flex';
}
function saveTotalEdit(){
  const newCal=parseFloat(gv('tot-cal').value);
  const newP  =parseFloat(gv('tot-p').value);
  const newC  =parseFloat(gv('tot-c').value);
  const newF  =parseFloat(gv('tot-f').value);
  if([newCal,newP,newC,newF].some(isNaN)){alert('Fill in all four fields.');retur
  const sel=ingredients.filter(i=>i.selected);
  if(!sel.length){closeTotalEdit();return;}
  // current totals (with multipliers)
  const cur=sel.reduce((a,i)=>({cal:a.cal+i.calories*i.portion_multiplier,p:a.p+i
  // scale ratio per macro — guard div-by-zero
  const rCal=cur.cal>0?newCal/cur.cal:0;
  const rP  =cur.p>0?newP/cur.p:0;
  const rC  =cur.c>0?newC/cur.c:0;
  const rF  =cur.f>0?newF/cur.f:0;
  // apply scaling directly to each selected ingredient's base values (reset mult
  sel.forEach(i=>{
    i.calories=Math.round(i.calories*i.portion_multiplier*rCal);
    i.protein =Math.round(i.protein *i.portion_multiplier*rP *10)/10;
    i.carbs   =Math.round(i.carbs   *i.portion_multiplier*rC *10)/10;
    i.fat     =Math.round(i.fat     *i.portion_multiplier*rF *10)/10;
    i.portion_multiplier=1;
```

```
    i.portion='custom';
  });
  closeTotalEdit();
  renderIngredients();
}

// INLINE IMPACT PANEL (inside log modal)
function renderLogImpact(){
  const el=gv('log-impact');if(!el)return;
  const sel=ingredients.filter(i=>i.selected);
  if(!sel.length){el.style.display='none';return;}
  const tot=sel.reduce((a,i)=>({cal:a.cal+i.calories*i.portion_multiplier,p:a.p+i
  const t=getTotals(),tgt=getCalTarget();
  const cal=Math.round(tot.cal),p=Math.round(tot.p*10)/10,c=Math.round(tot.c*10)/
  const remCal=Math.round(tgt-t.cal),remP=Math.round(TARGETS.p-t.p),remC=Math.rou

  // Score
  let score=0;
  if(remP>5&&p>0)score+=Math.min(p/remP,1)*35;
  if(remC>5&&c>0)score+=Math.min(c/remC,1)*25;
  if(remF>5&&f>0)score+=Math.min(f/remF,1)*20;
  if(remCal>50&&cal>0)score+=Math.min(cal/remCal,1)*20;
  if(p>remP+10)score-=15;
  if(cal>remCal+100)score-=20;
  score=Math.max(0,Math.min(100,Math.round(score)));
  let scoreLabel,scoreCls;
  if(score>=75){scoreLabel='Great fit 🔥';scoreCls='score-great';}
  else if(score>=50){scoreLabel='Good fit ✓';scoreCls='score-good';}
  else if(score>=25){scoreLabel='Okay fit';scoreCls='score-ok';}
  else{scoreLabel='Poor fit';scoreCls='score-poor';}
  gv('li-score').textContent=scoreLabel;
  gv('li-score').className=`impact-score ${scoreCls}`;

  // Remaining chips
  const chips=[
    {v:remCal,unit:'kcal',label:'left',cls:'cal'},
    {v:remP,unit:'g',label:'protein',cls:'p'},
    {v:remC,unit:'g',label:'carbs',cls:'c'},
    {v:remF,unit:'g',label:'fat',cls:'f'},
  ];
  gv('li-needs').innerHTML=chips.map(chip=>{
    const done=chip.v<=0;
    return `<div class="need-chip ${chip.cls}${done?' done':''}">${done?`✓ ${chip
  }).join('');

  // Macro bars
  const bars=[
```

```
    {lbl:'Protein',cur:t.p,  add:p,  tgt:TARGETS.p, col:'var(--pc)'},
    {lbl:'Carbs',  cur:t.c,  add:c,  tgt:TARGETS.c, col:'var(--cc)'},
    {lbl:'Fat',    cur:t.f,  add:f,  tgt:TARGETS.f, col:'var(--fc)'},
    {lbl:'Cals',   cur:t.cal,add:cal,tgt:tgt,        col:'var(--blue2)'},
  ];
  gv('li-bars').innerHTML=bars.map(b=>{
    const curPct=Math.min(b.cur/b.tgt*100,100);
    const addPct=Math.max(0,Math.min((b.cur+b.add)/b.tgt*100,100)-curPct);
    const after=b.lbl==='Cals'?Math.round(b.cur+b.add)+'kcal':Math.round(b.cur+b.
    const afterCol=(b.cur+b.add)>b.tgt*1.05?'var(--red)':b.col;
    return `<div class="ibar-row">
      <div class="ibar-lbl" style="color:${b.col}">${b.lbl}</div>
      <div class="ibar-track">
        <div style="position:absolute;top:0;left:0;height:100%;width:${curPct}%;b
        <div style="position:absolute;top:0;left:${curPct}%;height:100%;width:${a
      </div>
      <div class="ibar-after" style="color:${afterCol}">${after}</div>
    </div>`;
  }).join('');

  // Verdict: simple auto-generated based on score & macros
  let verdict='';
  if(score>=75)verdict=`Solid choice — fills ${Math.round(p/Math.max(remP,1)*100)
  else if(cal>remCal+100)verdict=`This puts you ${Math.round(cal-remCal)} kcal ov
  else if(p<remP*0.15)verdict=`Low protein hit — only ${p}g. Pair with a protein
  else verdict=`Decent option. Covers ${Math.round(cal/Math.max(remCal,1)*100)}%
  gv('li-verdict').textContent=verdict;

  el.style.display='block';
}

// ADD MISSING INGREDIENT
function toggleAddIngForm(){
  const f=gv('add-ing-form');
  f.style.display=f.style.display===='block'?'none':'block';
  if(f.style.display===='block'){gv('add-ing-name').focus();}
}
async function aiLookupIngredient(){
  const name=gv('add-ing-name').value.trim();if(!name){alert('Enter ingredient na
  gv('ing-lookup-loading').classList.add('show');
  try{
    const res=await fetch(PROXY,{method:'POST',headers:{'Content-Type':'applicati
      body:JSON.stringify({model:'claude-sonnet-4-20250514',max_tokens:300,
        system:'Return ONLY valid JSON, no markdown: {"emoji":"emoji","portion":"
        messages:[{role:'user',content:`Nutrition facts for: ${name}`}]})});
    const data=await res.json();
    const p=JSON.parse(data.content.map(b=>b.text||'').join('').replace(/```json|
```

```
    if(p.emoji)gv('add-ing-emoji').value=p.emoji;
    if(p.calories)gv('add-ing-cal').value=p.calories;
    if(p.protein)gv('add-ing-p').value=p.protein;
    if(p.carbs)gv('add-ing-c').value=p.carbs;
    if(p.fat)gv('add-ing-f').value=p.fat;
    if(p.portion)gv('add-ing-portion').value=p.portion;
  }catch(e){alert('Lookup failed. Fill in manually.');}
  finally{gv('ing-lookup-loading').classList.remove('show');}
}
function addManualIngredient(){
  const name=gv('add-ing-name').value.trim();if(!name){alert('Enter a name.');ret
  const newId=ingredients.length>0?Math.max(...ingredients.map(i=>i.id))+1:0;
  ingredients.push({id:newId,name,emoji:gv('add-ing-emoji').value||'🍽️',
    portion:gv('add-ing-portion').value||'1 serving',
    calories:parseFloat(gv('add-ing-cal').value)||0,
    protein:parseFloat(gv('add-ing-p').value)||0,
    carbs:parseFloat(gv('add-ing-c').value)||0,
    fat:parseFloat(gv('add-ing-f').value)||0,
    portion_multiplier:1,selected:true});
  ['add-ing-name','add-ing-emoji','add-ing-cal','add-ing-p','add-ing-c','add-ing-
  gv('add-ing-form').style.display='none';
  renderIngredients();
}

let _confirmLock=false;
function confirmMeal(){
  if(_confirmLock)return;
  const sel=ingredients.filter(i=>i.selected);
  if(!sel.length){alert('Select at least one ingredient.');return;}
  _confirmLock=true;
  const t=sel.reduce((a,i)=>({cal:a.cal+i.calories*i.portion_multiplier,p:a.p+i.p
  // Capture thumb BEFORE reset clears mealB64. Never persist to localStorage — b
  const thumbForSession=mealB64?`${mealB64}`:null;
  const entry={
    name:sel.length===1?sel[0].name:sel.map(i=>i.name).join(' + '),
    emoji:sel[0].emoji,
    calories:Math.round(t.cal),protein:Math.round(t.p*10)/10,
    carbs:Math.round(t.c*10)/10,fat:Math.round(t.f*10)/10,
    // extended nutrients from analysis if available
    fibre:Math.round((sel.reduce((a,i)=>(a+(i.fibre||0)*i.portion_multiplier),0))
    sugar:Math.round((sel.reduce((a,i)=>(a+(i.sugar||0)*i.portion_multiplier),0))
    sodium:Math.round(sel.reduce((a,i)=>(a+(i.sodium||0)*i.portion_multiplier),0)
    ingredients:sel.map(i=>({
      name:i.name,emoji:i.emoji,portion:i.portion,
      calories:Math.round(i.calories*i.portion_multiplier),
      protein:Math.round(i.protein*i.portion_multiplier*10)/10,
      carbs:Math.round(i.carbs*i.portion_multiplier*10)/10,
```

```
      fat:Math.round(i.fat*i.portion_multiplier*10)/10
    })),
    loggedAt:Date.now(),
    thumb:null
  };
  meals.push(entry);
  save(`${KEY}_meals_${todayKey()}`,meals);
  entry.thumb=thumbForSession; // attach to in-memory only
  gv('log-modal').classList.remove('open');
  resetLogModal();
  renderAll();
  const todayBtn=document.querySelector('.nb');
  if(todayBtn)showPage('today',todayBtn);
  setTimeout(()=>{_confirmLock=false;},800);
}

// AI COACH
function updateCoachStats(){
  const t=getTotals();
  gv('cs-cal').textContent=Math.round(t.cal)||'—';
  gv('cs-protein').textContent=Math.round(t.p)||'—';
  gv('cs-water').textContent=cups*ML_PER_CUP||'—';
}
// Day context string — reused for coach chat
function getDayContext(){
  const t=getTotals();
  const ws=whoopSnaps;
  const mealSum=meals.map(m=>`${m.name}: ${Math.round(m.calories)} kcal, ${Math.r
  const whoopSum=`Wake: Sleep ${ws[0]?.sleep??'—'}h, Recovery ${ws[0]?.recovery??
  return `BORNA'S DATA TODAY:\nMEALS:\n${mealSum||'None logged'}\nTOTALS: ${Math.
}

let chatHistory=[];

async function generateCoachReport(){
  if(!meals.length&&!whoopSnaps.some(s=>s!==null)){alert('Log some meals or Whoop
  gv('coach-loading').classList.add('show');gv('coach-response').classList.remove
  gv('coach-chat').classList.remove('show');
  chatHistory=[];
  const ctx=getDayContext();
  const prompt=`${ctx}\n\nGive me a direct daily debrief:\n1. OVERALL SCORE (1-10
  try{
    const res=await fetch(PROXY,{method:'POST',headers:{'Content-Type':'applicati
      body:JSON.stringify({model:'claude-sonnet-4-20250514',max_tokens:900,
        system:'You are a direct, no-nonsense performance and nutrition coach for
        messages:[{role:'user',content:prompt}]})});
    const data=await res.json();
```

```
    const text=data.content.map(b=>b.text||'').join('').trim();
    const re=gv('coach-response');
    re.innerHTML=`<div class="cr-header"><div class="cr-icon">🧠</div><div><div c
    re.classList.add('show');
    // Store debrief in chat history as context
    chatHistory=[
      {role:'user',content:prompt},
      {role:'assistant',content:text}
    ];
    // Show chat
    gv('chat-messages').innerHTML='';
    gv('coach-chat').classList.add('show');
  }catch(err){alert('Report failed.');console.error(err);}
  finally{gv('coach-loading').classList.remove('show');}
}

function handleChatKey(e){
  if(e.key==='Enter'&&!e.shiftKey){e.preventDefault();sendChatMessage();}
}

async function sendChatMessage(){
  const inp=gv('chat-input');
  const msg=inp.value.trim();if(!msg)return;
  inp.value='';inp.style.height='auto';

  const msgEl=gv('chat-messages');
  // Add user bubble
  const uDiv=document.createElement('div');
  uDiv.className='chat-msg user';uDiv.textContent=msg;
  msgEl.appendChild(uDiv);

  // Add loading bubble
  const lDiv=document.createElement('div');
  lDiv.className='chat-msg loading';lDiv.textContent='…';
  msgEl.appendChild(lDiv);
  msgEl.scrollTop=msgEl.scrollHeight;

  // Build messages — always include day context as system + history
  chatHistory.push({role:'user',content:msg});

  try{
    const res=await fetch(PROXY,{method:'POST',headers:{'Content-Type':'applicati
      body:JSON.stringify({model:'claude-sonnet-4-20250514',max_tokens:600,
        system:`You are a direct, no-nonsense performance and nutrition coach for
        messages:chatHistory})});
    const data=await res.json();
    const reply=data.content.map(b=>b.text||'').join('').trim();
```

```
    chatHistory.push({role:'assistant',content:reply});
    lDiv.className='chat-msg coach';
    lDiv.innerHTML=reply.replace(/\n/g,'<br>');
  }catch(err){
    lDiv.className='chat-msg coach';
    lDiv.textContent='Something went wrong. Try again.';
    chatHistory.pop();
  }
  msgEl.scrollTop=msgEl.scrollHeight;
}

// ── WELCOME CARD ──────────────────────────────────
const WMO_CODES={0:'Clear sky',1:'Mainly clear',2:'Partly cloudy',3:'Overcast',45
const WMO_EMOJI={0:'☀️',1:'🌤️',2:'⛅',3:'☁️',45:'🌫️',48:'🌫️',51:'🌦️',53:'🌦️',55

function getTimeOfDay(h){
  if(h>=5&&h<12)return'morning';
  if(h>=12&&h<17)return'afternoon';
  if(h>=17&&h<21)return'evening';
  return'night';
}

function getGreeting(h){
  const tod=getTimeOfDay(h);
  const greetings={morning:'Good morning, Borna 🌅',afternoon:'Good afternoon, Bc
  return greetings[tod];
}

function getSmartSub(){
  const t=getTotals();
  const tgt=getCalTarget();
  const rem=tgt-t.cal;
  const h=new Date().getHours();
  const hints=[];
  if(cups<4)hints.push('💧 You\'re behind on water — drink up');
  else if(cups>=8)hints.push('💧 Hydration on point today');
  if(t.p<TARGETS.p*0.5)hints.push(`🥩 Only ${Math.round(t.p)}g protein — hit thos
  else if(t.p>=TARGETS.p)hints.push(`🥩 Protein target crushed — ${Math.round(t.p
  if(rem>800&&h>=14)hints.push(`🍽️ ${Math.round(rem)} kcal left — don't skip meal
  else if(rem<0)hints.push(`⚠️ Over target by ${Math.abs(Math.round(rem))} kcal`)
  else if(rem<200)hints.push(`✅ Almost at your ${tgt} kcal target`);
  if(!meals.length&&h>=10)hints.push('🍽️ No meals logged yet today');
  if(whoopSnaps[0]?.recovery&&whoopSnaps[0].recovery<33)hints.push('😴 Low recove
  else if(whoopSnaps[0]?.recovery&&whoopSnaps[0].recovery>=67)hints.push(`💪 ${wh
  if(!hints.length)hints.push('Track your meals and Whoop data below');
  return hints[0];
}
```

```
// ── WELCOME CARD LIVE SCENE ──────────────────────────────────────────────
let _wcRaf=null;
let _wcScene=null;

function _wcStop(){if(_wcRaf){cancelAnimationFrame(_wcRaf);_wcRaf=null;}}

function _wcInitScene(tod,w,h){
  // Seeded pseudo-random so layout is stable
  const rng=(s)=>{let x=Math.sin(s)*10000;return x-Math.floor(x);};
  // Stars
  const stars=Array.from({length:32},(_,i)=>({
    x:rng(i*7+1)*w, y:rng(i*13+2)*h*0.8,
    r:rng(i*3+3)<0.25?1.3:0.6,
    phase:rng(i*11+4)*Math.PI*2,
    speed:0.4+rng(i*5+5)*0.8
  }));
  // Clouds — 2 per scene, different speeds/sizes
  const clouds=[
    {x:w*0.55, y:h*0.22, scale:1,   speed:0.012, opacity:0.18},
    {x:w*0.2,  y:h*0.40, scale:0.7, speed:0.007, opacity:0.12},
  ];
  return {tod,w,h,t:0,stars,clouds,
    sunX:w*0.82, sunY:h*0.30,
    moonX:w*0.78, moonY:h*0.22,
  };
}

function _drawCloud(ctx,cx,cy,scale,alpha){
  ctx.save();
  ctx.globalAlpha=alpha;
  ctx.fillStyle='rgba(255,255,255,1)';
  const s=scale;
  // Simple 3-circle cloud
  const blob=(x,y,r)=>{ctx.beginPath();ctx.arc(cx+x*s,cy+y*s,r*s,0,Math.PI*2);ctx
  blob(0,0,11); blob(-13,-4,8); blob(13,-4,9); blob(-22,4,6); blob(22,4,6);
  ctx.restore();
}

function _wcFrame(canvas,bg,scene){
  const {tod,w,h,t}=scene;
  const dpr=scene.dpr||1;
  const ctx=canvas.getContext('2d');
  ctx.setTransform(dpr,0,0,dpr,0,0);
  ctx.clearRect(0,0,w,h);

  if(tod==='morning'){
```

```
    // Sky gradient
    const sk=ctx.createLinearGradient(0,0,w,h);
    sk.addColorStop(0,'#0d2545'); sk.addColorStop(1,'#1e3d20');
    ctx.fillStyle=sk; ctx.fillRect(0,0,w,h);
    // Sun glow
    const sx=scene.sunX, sy=scene.sunY;
    const pulse=1+Math.sin(t*0.9)*0.04;
    const glow=ctx.createRadialGradient(sx,sy,0,sx,sy,w*0.5*pulse);
    glow.addColorStop(0,'rgba(255,200,80,0.22)');
    glow.addColorStop(0.4,'rgba(255,160,40,0.10)');
    glow.addColorStop(1,'transparent');
    ctx.fillStyle=glow; ctx.fillRect(0,0,w,h);
    // Rotating rays
    ctx.save(); ctx.translate(sx,sy);
    const numRays=12;
    for(let i=0;i<numRays;i++){
      const angle=(i/numRays)*Math.PI*2+t*0.25;
      const len=18+Math.sin(t*1.1+i)*3;
      const a=0.06+Math.sin(t*0.7+i*0.5)*0.02;
      ctx.strokeStyle=`rgba(255,210,80,${a})`;
      ctx.lineWidth=2.5;
      ctx.beginPath();
      ctx.moveTo(Math.cos(angle)*14,Math.sin(angle)*14);
      ctx.lineTo(Math.cos(angle)*(14+len),Math.sin(angle)*(14+len));
      ctx.stroke();
    }
    ctx.restore();
    // Sun circle
    const sunG=ctx.createRadialGradient(sx,sy,0,sx,sy,12);
    sunG.addColorStop(0,'rgba(255,240,160,0.95)');
    sunG.addColorStop(1,'rgba(255,190,60,0.80)');
    ctx.fillStyle=sunG;
    ctx.beginPath(); ctx.arc(sx,sy,12,0,Math.PI*2); ctx.fill();
    // Drifting clouds
    scene.clouds.forEach(c=>{
      c.x=(c.x+c.speed)%(w+80)-40;
      _drawCloud(ctx,c.x,c.y,c.scale,c.opacity+Math.sin(t*0.3+c.x*0.01)*0.03);
    });

  } else if(tod==='afternoon'){
    // Deeper blue-green sky
    const sk=ctx.createLinearGradient(0,0,w,h);
    sk.addColorStop(0,'#071a2e'); sk.addColorStop(1,'#0d2416');
    ctx.fillStyle=sk; ctx.fillRect(0,0,w,h);
    const sx=scene.sunX-w*0.1, sy=h*0.18;
    // Strong glow
    const pulse=1+Math.sin(t*0.6)*0.03;
```

```
    const glow=ctx.createRadialGradient(sx,sy,0,sx,sy,w*0.55*pulse);
    glow.addColorStop(0,'rgba(245,200,60,0.25)');
    glow.addColorStop(0.35,'rgba(245,160,30,0.10)');
    glow.addColorStop(1,'transparent');
    ctx.fillStyle=glow; ctx.fillRect(0,0,w,h);
    // Heat shimmer rays — longer, slower
    ctx.save(); ctx.translate(sx,sy);
    for(let i=0;i<8;i++){
      const angle=(i/8)*Math.PI*2+t*0.12;
      const len=22+Math.sin(t*0.5+i*0.8)*5;
      ctx.strokeStyle=`rgba(255,220,60,${0.05+Math.sin(t*0.4+i)*0.02})`;
      ctx.lineWidth=3;
      ctx.beginPath();
      ctx.moveTo(Math.cos(angle)*15,Math.sin(angle)*15);
      ctx.lineTo(Math.cos(angle)*(15+len),Math.sin(angle)*(15+len));
      ctx.stroke();
    }
    ctx.restore();
    // Sun
    const sunG=ctx.createRadialGradient(sx,sy,0,sx,sy,13);
    sunG.addColorStop(0,'rgba(255,248,180,0.98)');
    sunG.addColorStop(1,'rgba(245,200,40,0.85)');
    ctx.fillStyle=sunG;
    ctx.beginPath(); ctx.arc(sx,sy,13,0,Math.PI*2); ctx.fill();
    // One slow cloud
    const c=scene.clouds[0];
    c.x=(c.x+c.speed*0.7)%(w+80)-40;
    _drawCloud(ctx,c.x,c.y*0.6,c.scale*1.2,c.opacity*0.8);

  } else if(tod==='evening'){
    // Deep dusk
    const sk=ctx.createLinearGradient(0,0,0,h);
    sk.addColorStop(0,'#1a0828'); sk.addColorStop(0.6,'#2d0f0a'); sk.addColorStop
    ctx.fillStyle=sk; ctx.fillRect(0,0,w,h);
    // Horizon glow
    const sx=scene.sunX-w*0.05, sy=h*0.88;
    const hglow=ctx.createRadialGradient(sx,sy,0,sx,sy,w*0.7);
    hglow.addColorStop(0,'rgba(255,100,30,0.28)');
    hglow.addColorStop(0.5,'rgba(200,50,10,0.10)');
    hglow.addColorStop(1,'transparent');
    ctx.fillStyle=hglow; ctx.fillRect(0,0,w,h);
    // Top ambient
    const aglow=ctx.createRadialGradient(w*0.72,h*0.1,0,w*0.72,h*0.1,w*0.4);
    const aPulse=0.08+Math.sin(t*0.4)*0.02;
    aglow.addColorStop(0,`rgba(180,60,180,${aPulse})`);
    aglow.addColorStop(1,'transparent');
    ctx.fillStyle=aglow; ctx.fillRect(0,0,w,h);
```

```
    // Few stars appearing
    scene.stars.slice(0,12).forEach(s=>{
      const twinkle=0.3+Math.sin(t*s.speed+s.phase)*0.3;
      ctx.fillStyle=`rgba(255,255,255,${twinkle})`;
      ctx.beginPath(); ctx.arc(s.x,s.y*0.7,s.r*0.8,0,Math.PI*2); ctx.fill();
    });
    // Slow cloud silhouette
    scene.clouds.forEach(c=>{
      c.x=(c.x+c.speed*0.5)%(w+80)-40;
      ctx.save();
      ctx.globalAlpha=0.07;
      ctx.fillStyle='rgba(255,120,60,1)';
      const blob=(x,y,r)=>{ctx.beginPath();ctx.arc(c.x+x,c.y+y,r,0,Math.PI*2);ctx
      const s=c.scale;
      blob(0,0,11*s); blob(-13,-4,8*s); blob(13,-4,9*s); blob(-22,4,6*s); blob(22
      ctx.restore();
    });

  } else { // night
    const sk=ctx.createLinearGradient(0,0,w,h);
    sk.addColorStop(0,'#03050d'); sk.addColorStop(1,'#060810');
    ctx.fillStyle=sk; ctx.fillRect(0,0,w,h);
    // Stars twinkling
    scene.stars.forEach(s=>{
      const twinkle=0.4+Math.sin(t*s.speed+s.phase)*0.35;
      ctx.fillStyle=`rgba(255,255,255,${twinkle})`;
      ctx.beginPath(); ctx.arc(s.x,s.y,s.r,0,Math.PI*2); ctx.fill();
      // Occasional sparkle cross
      if(s.r>1){
        const spark=Math.max(0,Math.sin(t*s.speed*1.5+s.phase)-0.5)*0.3;
        if(spark>0){
          ctx.strokeStyle=`rgba(200,220,255,${spark})`;
          ctx.lineWidth=0.5;
          ctx.beginPath();ctx.moveTo(s.x-4,s.y);ctx.lineTo(s.x+4,s.y);ctx.stroke(
          ctx.beginPath();ctx.moveTo(s.x,s.y-4);ctx.lineTo(s.x,s.y+4);ctx.stroke(
        }
      }
    });
    // Crescent moon — slowly drifting
    const mx=scene.moonX+Math.sin(t*0.05)*6;
    const my=scene.moonY+Math.cos(t*0.04)*3;
    const moonGlow=ctx.createRadialGradient(mx,my,0,mx,my,28);
    moonGlow.addColorStop(0,'rgba(180,200,255,0.10)');
    moonGlow.addColorStop(1,'transparent');
    ctx.fillStyle=moonGlow; ctx.fillRect(0,0,w,h);
    // Moon body
    ctx.fillStyle='rgba(210,225,255,0.90)';
```

```
    ctx.beginPath(); ctx.arc(mx,my,10,0,Math.PI*2); ctx.fill();
    // Bite out for crescent
    ctx.globalCompositeOperation='destination-out';
    ctx.fillStyle='rgba(0,0,0,1)';
    ctx.beginPath(); ctx.arc(mx+5,my-2,8,0,Math.PI*2); ctx.fill();
    ctx.globalCompositeOperation='source-over';
    // Subtle blue ambient
    const bglow=ctx.createRadialGradient(w*0.8,h*0.3,0,w*0.8,h*0.3,w*0.45);
    bglow.addColorStop(0,'rgba(40,70,160,0.06)');
    bglow.addColorStop(1,'transparent');
    ctx.fillStyle=bglow; ctx.fillRect(0,0,w,h);
  }

  scene.t+=0.016; // ~60fps time increment
  _wcRaf=requestAnimationFrame(()=>_wcFrame(canvas,bg,scene));
}

function setBgForTime(h,weatherCode){
  _wcStop(); // cancel any existing loop
  const canvas=gv('wc-canvas');
  const bg=gv('wc-bg');
  if(!canvas||!bg)return;
  // Size canvas to physical pixels for crisp rendering
  const dpr=Math.min(window.devicePixelRatio||1,2);
  const pw=canvas.parentElement.offsetWidth||canvas.parentElement.clientWidth||36
  const ph=(canvas.parentElement.offsetHeight||canvas.parentElement.clientHeight)
  canvas.width=pw*dpr;
  canvas.height=ph*dpr;
  canvas.style.width=pw+'px';
  canvas.style.height=ph+'px';
  const tod=getTimeOfDay(h);
  const bgs={
    morning:  'linear-gradient(135deg,#0c2240,#1a3520)',
    afternoon:'linear-gradient(135deg,#08172e,#0c2818)',
    evening:  'linear-gradient(135deg,#1c0a30,#280e0e)',
    night:    'linear-gradient(135deg,#04060e,#080816)',
  };
  bg.style.background=bgs[tod];
  _wcScene=_wcInitScene(tod,pw,ph);
  _wcScene.dpr=dpr;
  // Pause loop when tab hidden (battery saving)
  document.removeEventListener('visibilitychange',_wcVisChange);
  document.addEventListener('visibilitychange',_wcVisChange);
  _wcRaf=requestAnimationFrame(()=>_wcFrame(canvas,bg,_wcScene));
}
function _wcVisChange(){
  if(document.hidden){_wcStop();}
```

```
  else if(_wcScene){
    const canvas=gv('wc-canvas');
    const bg=gv('wc-bg');
    if(canvas&&bg)_wcRaf=requestAnimationFrame(()=>_wcFrame(canvas,bg,_wcScene));
  }
}

async function initWelcomeCard(){
  const h=new Date().getHours();
  gv('wc-greeting').textContent=getGreeting(h);
  gv('wc-sub').textContent=getSmartSub();
  // Defer until browser has painted and card has real dimensions
  setTimeout(()=>setBgForTime(h,0), 60);
  // Fetch Dubai weather from Open-Meteo (no API key needed)
  try{
    const r=await fetch('https://api.open-meteo.com/v1/forecast?latitude=25.2048&
    const d=await r.json();
    const temp=Math.round(d.current.temperature_2m);
    const code=d.current.weathercode;
    const desc=WMO_CODES[code]||'Dubai';
    const emoji=WMO_EMOJI[code]||'🌡';
    gv('wc-wicon').textContent=emoji;
    gv('wc-wtemp').textContent=`${temp}°C`;
    gv('wc-wdesc').textContent=desc+' · Dubai';
    gv('wc-weather').style.display='flex';
    setBgForTime(h,code);
  }catch(e){
    // Weather failed silently — card still shows
  }
}



// CALENDAR
function buildCalendar(){
  const ce=gv('cal-grid');if(!ce)return;
  const y=calViewDate.getFullYear(),m=calViewDate.getMonth();
  gv('cal-month-lbl').textContent=MONTHS[m]+' '+y;
  gv('cal-dow').innerHTML=DOWS.map(d=>`<div class="cal-dow">${d}</div>`).join('')
  const fd=new Date(y,m,1).getDay(),days=new Date(y,m+1,0).getDate(),ts=todayKey(
  let html='';
  for(let i=0;i<fd;i++)html+=`<div class="cc empty"></div>`;
  for(let d=1;d<=days;d++){
    const ds=`${y}-${String(m+1).padStart(2,'0')}-${String(d).padStart(2,'0')}`;
    const hm=load(`${KEY}_meals_${ds}`,[]).length>0;
    const hw=load(`${KEY}_whoopsnaps_${ds}`,[null,null,null]).some(s=>s!==null);
    html+=`<div class="cc${ds===ts?' today':''}${ds===calSelKey?' sel':''}${(hm||
```

```
  }
  ce.innerHTML=html;
  renderDayDetail(calSelKey);
}
function changeMonth(dir){calViewDate.setMonth(calViewDate.getMonth()+dir);buildC
function selectDay(ds){calSelKey=ds;buildCalendar();}
function renderDayDetail(ds){
  const el=gv('day-detail');if(!el)return;
  const dm=load(`${KEY}_meals_${ds}`,[]),dw=load(`${KEY}_whoopsnaps_${ds}`,[null,
  const dc=parseInt(localStorage.getItem(`${KEY}_cups_${ds}`)||'0');
  const t=getTotals(dm);
  const d=new Date(ds+'T12:00:00');
  const lbl=d.toLocaleDateString('en-US',{weekday:'long',month:'long',day:'numeri
  let html=`<div class="ddc">
    <div class="ddc-title">${lbl}</div>
    <div class="ddc-row" style="grid-template-columns:repeat(5,1fr)">
      <div class="ddc-s"><div class="ddc-v" style="color:var(--blue2)">${t.cal>0?
      <div class="ddc-s"><div class="ddc-v" style="color:var(--pc)">${t.p>0?Math.
      <div class="ddc-s"><div class="ddc-v" style="color:var(--cc)">${t.c>0?Math.
      <div class="ddc-s"><div class="ddc-v" style="color:var(--fc)">${t.f>0?Math.
      <div class="ddc-s"><div class="ddc-v" style="color:var(--wc)">${dc>0?dc:'—'
    </div>
  </div>`;
  const snapLabels=['Wake','1PM','EOD'];
  dw.forEach((s,i)=>{
    if(!s)return;
    const cfg=SNAP_CFG[i];
    const items=cfg.fields.map(f=>{
      const v=s[f.id];if(v==null)return'';
  const disp=f.id==='steps'?Number(v).toLocaleString():f.id==='recovery'?v+'%':f.
      return`<div class="ddc-s"><div class="ddc-v" style="color:var(--whoop-green
    }).filter(Boolean).join('');
    if(items)html+=`<div class="ddc"><div class="ddc-title" style="color:var(--wh
  });
  if(dm.length){
    html+=`<div class="ddc"><div class="ddc-title">Meals</div>`;
    dm.forEach(m=>html+=`<div class="dm-item"><div class="dm-name">${m.emoji||'🍽
    html+=`</div>`;
  }else html+=`<div class="no-data">No meals logged on this day.</div>`;
  el.innerHTML=html;
}

// PROGRESS
function openEntryModal(){gv('entry-modal').classList.add('open');}
function closeEntryModal(){gv('entry-modal').classList.remove('open');}
function saveEntry(){
  const w=gv('e-w').value!==''?parseFloat(gv('e-w').value):null;
```

```
  const bf=gv('e-bf').value!==''?parseFloat(gv('e-bf').value):null;
  if(!w&&!bf){alert('Enter weight or body fat %.');return;}
  entries.push({date:new Date().toISOString(),weight:w,bf,notes:gv('e-notes').val
  save(`${KEY}_entries`,entries);
  gv('e-w').value='';gv('e-bf').value='';gv('e-notes').value='';
  closeEntryModal();renderProgress();
}
function deleteEntry(i){entries.splice(i,1);save(`${KEY}_entries`,entries);render
function renderProgress(){
  const wEs=entries.filter(e=>e.weight!=null),bEs=entries.filter(e=>e.bf!=null);
  const lw=wEs.length?wEs[wEs.length-1].weight:null,lb=bEs.length?bEs[bEs.length-
  const pw=wEs.length>1?wEs[wEs.length-2].weight:BASELINE.weight,pb=bEs.length>1?
  gv('p-weight').textContent=lw!=null?lw.toFixed(1):BASELINE.weight.toFixed(1);
  gv('p-bf').textContent=lb!=null?lb.toFixed(1):BASELINE.bf.toFixed(1);
  const wd=lw!=null?(lw-pw).toFixed(1):null,bd=lb!=null?(lb-pb).toFixed(1):null;
  if(wd)gv('p-w-delta').innerHTML=`<span class="${parseFloat(wd)<0?'dg':'db'}">${
  if(bd)gv('p-bf-delta').innerHTML=`<span class="${parseFloat(bd)<0?'dg':'db'}">$
  const curBF=lb||BASELINE.bf,curW=lw||BASELINE.weight;
  const curFat=(curBF/100)*curW,tgtFat=(GOAL_BF/100)*BASELINE.weight;
  const fatLost=BASELINE.fatMass-curFat,totalLose=BASELINE.fatMass-tgtFat;
  const pct=Math.max(0,Math.min(100,Math.round((fatLost/totalLose)*100)));
  gv('goal-pct').textContent=pct+'%';gv('gbar-f').style.width=pct+'%';
  gv('goal-detail').textContent=`Baseline: 22.4kg fat → Target: ${tgtFat.toFixed(
  renderChart('chart-w',wEs.slice(-8),'weight','kg','#388bfd',84,92);
  renderChart('chart-bf',bEs.slice(-8),'bf','%','#2dd4c8',18,28);
  const le=gv('entries-list');
  if(!entries.length){le.innerHTML='<div style="text-align:center;padding:24px;co
  let html='';
  [...entries].reverse().forEach((e,ri)=>{
    const i=entries.length-1-ri;
    const ds=new Date(e.date).toLocaleDateString('en-US',{month:'short',day:'nume
    html+=`<div class="ei"><div><div class="ei-date">${ds}</div>${e.notes?`<div c
      <div class="ei-vals">${e.weight!=null?`<div class="ei-v"><div class="ei-vn"
      ${e.bf!=null?`<div class="ei-v"><div class="ei-vn" style="color:var(--cyan)
      <button class="ei-del" onclick="deleteEntry(${i})">×</button></div></div>`;
  });
  le.innerHTML=html;
}
function renderChart(id,data,field,unit,color,minV,maxV){
  const el=gv(id);
  if(!data.length){el.innerHTML=`<div style="color:var(--muted);font-size:11px;pa
  const range=maxV-minV||1;let html='';
  data.forEach(e=>{
    const v=e[field],h=Math.max(6,Math.min(95,((v-minV)/range)*90));
    const dl=new Date(e.date).toLocaleDateString('en-US',{month:'short',day:'nume
    html+=`<div class="cb-wrap"><div class="cb" style="height:${h}px;background:$
  });
```

```
  el.innerHTML=html;
}

// ═══════════════════════════════════════════════════════════════════════
// FEATURE: NUTRIENT BREAKDOWN — shown below macro rings on Today tab
// ═══════════════════════════════════════════════════════════════════════
function renderNutrients(){
  const t=getTotals();
  const fibre=t.fibre||0, sugar=t.sugar||0, sodium=t.sodium||0;
  const el=gv('nutrient-breakdown');
  if(!el)return;
  const fibreW=Math.min(fibre/30,1), sugarW=Math.min(sugar/50,1), sodiumW=Math.mi
  const fibreCol=fibre>=25?'var(--green)':fibre>=15?'var(--amber)':'var(--muted)'
  const sugarCol=sugar>60?'var(--red)':sugar>40?'var(--amber)':'var(--cc)';
  const sodiumCol=sodium>2300?'var(--red)':sodium>1500?'var(--amber)':'var(--cyan
  el.innerHTML=`
    <div class="nut-row">
      <div class="nut-item macro-tap" onclick="openMacroBreakdown('fibre')">
        <div class="nut-top"><span class="nut-lbl">Fibre</span><span class="nut-v
        <div class="nut-bar-bg"><div class="nut-bar-f" style="width:${fibreW*100}
        <div class="nut-sub">target 30g</div>
      </div>
      <div class="nut-item macro-tap" onclick="openMacroBreakdown('sugar')">
        <div class="nut-top"><span class="nut-lbl">Sugar</span><span class="nut-v
        <div class="nut-bar-bg"><div class="nut-bar-f" style="width:${sugarW*100}
        <div class="nut-sub">keep under 50g</div>
      </div>
      <div class="nut-item macro-tap" onclick="openMacroBreakdown('sodium')">
        <div class="nut-top"><span class="nut-lbl">Sodium</span><span class="nut-
        <div class="nut-bar-bg"><div class="nut-bar-f" style="width:${sodiumW*100
        <div class="nut-sub">limit 2300mg</div>
      </div>
    </div>`;
}

// ═══════════════════════════════════════════════════════════════════════
// MACRO CONTRIBUTOR BREAKDOWN
// ═══════════════════════════════════════════════════════════════════════
const MACRO_CFG={
  calories: {label:'Calories',    unit:'kcal', key:'calories', color:'var(--blue2
  protein:  {label:'Protein',     unit:'g',    key:'protein',  color:'var(--pc)',
  carbs:    {label:'Carbs',       unit:'g',    key:'carbs',    color:'var(--cc)',
  fat:      {label:'Fat',         unit:'g',    key:'fat',      color:'var(--fc)',
  fibre:    {label:'Fibre',       unit:'g',    key:'fibre',    color:'var(--green
  sugar:    {label:'Sugar',       unit:'g',    key:'sugar',    color:'var(--amber
  sodium:   {label:'Sodium',      unit:'mg',   key:'sodium',   color:'var(--cyan)
};
```

```
function openMacroBreakdown(macro){
  const cfg=MACRO_CFG[macro];if(!cfg)return;
  // update config target for calories dynamically
  if(macro==='calories')cfg.target=getCalTarget();

  const modal=gv('macro-breakdown-modal');
  gv('mbd-title').textContent=cfg.label+' Contributors';

  // build ranked list from today's meals
  const ranked=meals
    .map((m,i)=>({idx:i, name:m.name, emoji:m.emoji||'🍽', val:m[cfg.key]||0}))
    .filter(r=>r.val>0)
    .sort((a,b)=>b.val-a.val);

  const total=ranked.reduce((s,r)=>s+r.val,0);
  const tgt=cfg.target;
  const pctOfTarget=tgt>0?Math.round(total/tgt*100):null;

  // sub-header
  let sub=cfg.fmt(total)+' total';
  if(pctOfTarget!==null) sub+=` · ${pctOfTarget}% of ${tgt}${cfg.unit} target`;
  gv('mbd-sub').textContent=sub;

  const listEl=gv('mbd-list');
  const emptyEl=gv('mbd-empty');

  if(!ranked.length){
    listEl.innerHTML='';
    emptyEl.style.display='block';
    modal.classList.add('open');
    return;
  }
  emptyEl.style.display='none';

  const maxVal=ranked[0].val;
  listEl.innerHTML=ranked.map((r,i)=>{
    const barPct=Math.round(r.val/maxVal*100);
    const sharePct=total>0?Math.round(r.val/total*100):0;
    return `<div class="mbd-row">
      <div class="mbd-rank">${i+1}</div>
      <div class="mbd-emoji">${r.emoji}</div>
      <div class="mbd-info">
        <div class="mbd-name">${r.name}</div>
        <div class="mbd-bar-wrap">
          <div class="mbd-bar-track">
            <div class="mbd-bar-fill" style="width:${barPct}%;background:${cfg.co
```

```
          </div>
          <span class="mbd-share">${sharePct}%</span>
        </div>
      </div>
      <div class="mbd-val" style="color:${cfg.color}">${cfg.fmt(r.val)}</div>
    </div>`;
  }).join('');

  modal.classList.add('open');
}
function closeMacroBreakdown(){
  const modal=gv('macro-breakdown-modal');
  if(modal)modal.classList.remove('open');
}

// ═══════════════════════════════════════════════════════════════════════════
// FEATURE: BARCODE SCANNER — OCR via Claude (works on all devices) + Open Food F
// ═══════════════════════════════════════════════════════════════════════════
let barcodeStream=null;
let _barcodeScanning=false;

function openBarcodeModal(){
  gv('barcode-modal').classList.add('open');
  gv('barcode-result').style.display='none';
  gv('barcode-manual').value='';
  _barcodeScanning=false;
  setBarcodeStatus('idle');
  startBarcodeCamera();
}
function closeBarcodeModal(){
  stopBarcodeCamera();
  _barcodeScanning=false;
  gv('barcode-modal').classList.remove('open');
}
function stopBarcodeCamera(){
  if(barcodeStream){barcodeStream.getTracks().forEach(t=>t.stop());barcodeStream=
  const v=gv('barcode-video');if(v)v.srcObject=null;
}
function setBarcodeStatus(state,txt){
  const el=gv('barcode-status');
  const btn=gv('barcode-snap-btn');
  if(state==='idle'){
    el.textContent='Aim at barcode then tap the button';
    if(btn){btn.disabled=false;btn.textContent='📷 Read Barcode';}
  } else if(state==='reading'){
    el.textContent='Reading barcode…';
    if(btn){btn.disabled=true;btn.textContent='Reading…';}
```

```
  } else if(state==='looking'){
    el.textContent='✓ Got number — looking up product…';
    if(btn)btn.disabled=true;
  } else if(state==='found'){
    el.textContent='✓ Product found';
    if(btn){btn.disabled=false;btn.textContent='📷 Scan Again';}
  } else if(state==='error'){
    el.textContent=txt||'Try again';
    if(btn){btn.disabled=false;btn.textContent='📷 Try Again';}
  }
}

async function startBarcodeCamera(){
  const video=gv('barcode-video');
  if(!video)return;
  try{
    barcodeStream=await navigator.mediaDevices.getUserMedia({
      video:{facingMode:'environment',width:{ideal:1920},height:{ideal:1080}}
    });
    video.srcObject=barcodeStream;
    video.play();
    setBarcodeStatus('idle');
  }catch(e){
    setBarcodeStatus('error','Camera unavailable. Type barcode number below.');
  }
}

// Capture a frame from video and send to Claude to read the barcode digits
async function snapAndReadBarcode(){
  if(_barcodeScanning)return;
  const video=gv('barcode-video');
  if(!video||!barcodeStream){
    // No camera — fall back to manual
    const code=gv('barcode-manual').value.trim();
    if(code)lookupBarcode(code);
    else setBarcodeStatus('error','No camera. Type the barcode number below.');
    return;
  }
  _barcodeScanning=true;
  setBarcodeStatus('reading');

  // Draw current video frame to canvas
  const canvas=document.createElement('canvas');
  canvas.width=video.videoWidth||1280;
  canvas.height=video.videoHeight||720;
  const ctx=canvas.getContext('2d');
  ctx.drawImage(video,0,0,canvas.width,canvas.height);
```

```
  const b64=canvas.toDataURL('image/jpeg',0.92).split(',')[1];

  try{
    const res=await fetch(PROXY,{method:'POST',headers:{'Content-Type':'applicati
      body:JSON.stringify({model:'claude-sonnet-4-20250514',max_tokens:100,
        system:'You are a barcode reader. Look at the image and find the barcode
        messages:[{role:'user',content:[
          {type:'image',source:{type:'base64',media_type:'image/jpeg',data:b64}},
          {type:'text',text:'What is the barcode number in this image? Return onl
        ]}]})});
    const data=await res.json();
    const raw=(data.content||[]).map(b=>b.text||'').join('').trim().replace(/\s/g
    // Extract only digits
    const digits=raw.replace(/[^0-9]/g,'');
    if(!digits||raw==='NONE'||digits.length<6){
      _barcodeScanning=false;
      setBarcodeStatus('error','Could not read barcode — reposition and try again
      return;
    }
    gv('barcode-manual').value=digits;
    setBarcodeStatus('looking');
    lookupBarcode(digits);
  }catch(e){
    _barcodeScanning=false;
    setBarcodeStatus('error','Read failed — try again.');
  }
}

async function lookupBarcode(code){
  if(!code||code.trim()===''){alert('Enter a barcode number.');return;}
  gv('barcode-status').textContent='Looking up…';
  gv('barcode-result').style.display='none';
  try{
    const res=await fetch(`https://world.openfoodfacts.org/api/v2/product/${code.
    const data=await res.json();
    if(data.status!==1||!data.product){
      _barcodeScanning=false; setBarcodeStatus('error','Product not found in data
      return;
    }
    const p=data.product;
    const n=p.nutriments||{};
    // Per 100g values
    const per100={
      calories:Math.round(n['energy-kcal_100g']||n['energy_100g']/4.184||0),
      protein:Math.round((n['proteins_100g']||0)*10)/10,
      carbs:Math.round((n['carbohydrates_100g']||0)*10)/10,
      fat:Math.round((n['fat_100g']||0)*10)/10,
```

```
      fibre:Math.round((n['fiber_100g']||0)*10)/10,
      sugar:Math.round((n['sugars_100g']||0)*10)/10,
      sodium:Math.round((n['sodium_100g']||0)*1000)
    };
    const servingG=parseFloat(p.serving_quantity)||100;
    const factor=servingG/100;
    const serving={
      calories:Math.round(per100.calories*factor),
      protein:Math.round(per100.protein*factor*10)/10,
      carbs:Math.round(per100.carbs*factor*10)/10,
      fat:Math.round(per100.fat*factor*10)/10,
      fibre:Math.round(per100.fibre*factor*10)/10,
      sugar:Math.round(per100.sugar*factor*10)/10,
      sodium:Math.round(per100.sodium*factor)
    };
    const name=`${p.brands?p.brands.split(',')[0].trim()+' ':''}${p.product_name|
    const servingLabel=p.serving_size||`${servingG}g`;
    // Store for add
    window._barcodeEntry={name,emoji:'📦',...serving,thumb:null};
    window._barcodePer100=per100;
    window._barcodeServingG=servingG;
    window._barcodeName=name;
    gv('bc-name').textContent=name;
    gv('bc-serving').textContent=`Per serving (${servingLabel})`;
    gv('bc-cal').textContent=serving.calories+' kcal';
    gv('bc-p').textContent=serving.protein+'g P';
    gv('bc-c').textContent=serving.carbs+'g C';
    gv('bc-f').textContent=serving.fat+'g F';
    gv('bc-fibre').textContent=serving.fibre+'g fibre';
    gv('bc-sugar').textContent=serving.sugar+'g sugar';
    gv('bc-sodium').textContent=serving.sodium+'mg sodium';
    gv('barcode-qty').value='1';
    gv('barcode-result').style.display='block';
    _barcodeScanning=false;
    stopBarcodeCamera();
    setBarcodeStatus('found');
  }catch(e){
    _barcodeScanning=false;
    setBarcodeStatus('error','Lookup failed — product may not be in database.');
    console.error(e);
  }
}

let _barcodeLock=false;
function addBarcodeItem(){
  if(!window._barcodeEntry||_barcodeLock)return;
  _barcodeLock=true;
```

```
  const qty=parseFloat(gv('barcode-qty').value)||1;
  const e=window._barcodeEntry;
  const entry={
    name:`${e.name}${qty!==1?' ×'+qty:''}`,
    emoji:'◦',
    calories:Math.round(e.calories*qty),
    protein:Math.round(e.protein*qty*10)/10,
    carbs:Math.round(e.carbs*qty*10)/10,
    fat:Math.round(e.fat*qty*10)/10,
    fibre:Math.round(e.fibre*qty*10)/10,
    sugar:Math.round(e.sugar*qty*10)/10,
    sodium:Math.round(e.sodium*qty),
    thumb:null
  };
  meals.push(entry);
  save(`${KEY}_meals_${todayKey()}`,meals);
  renderAll();
  closeBarcodeModal();
  const todayBtn=document.querySelector('.nb');
  if(todayBtn)showPage('today',todayBtn);
  setTimeout(()=>{_barcodeLock=false;},800);
}



// ═══════════════════════════════════════════════════════════════════════
// FEATURE: MACRO IMPACT SCANNER
// ═══════════════════════════════════════════════════════════════════════
let impactB64=null;
let impactEntry=null;

function openImpactModal(){
  impactB64=null; impactEntry=null;
  gv('impact-desc').value='';
  gv('impact-result').classList.remove('show');
  gv('impact-loading').classList.remove('show');
  gv('impact-preview').style.display='none';
  gv('impact-ph-ph').style.display='block';
  gv('impact-photo-zone').classList.remove('has-img');
  gv('file-impact').value='';
  gv('impact-scan-btn').disabled=false;
  renderImpactNeeds();
  gv('impact-modal').classList.add('open');
}
function closeImpactModal(){ gv('impact-modal').classList.remove('open'); }

function renderImpactNeeds(){
```

```
  const t=getTotals(), tgt=getCalTarget();
  const chips=[
    {v:Math.round(tgt-t.cal), unit:'kcal', label:'left', cls:'cal'},
    {v:Math.round(TARGETS.p-t.p), unit:'g', label:'protein', cls:'p'},
    {v:Math.round(TARGETS.c-t.c), unit:'g', label:'carbs', cls:'c'},
    {v:Math.round(TARGETS.f-t.f), unit:'g', label:'fat', cls:'f'},
  ];
  gv('impact-needs').innerHTML=chips.map(chip=>{
    const done=chip.v<=0;
    const display=done?`✓ ${chip.label}`:`${chip.v}${chip.unit} ${chip.label} lef
    return `<div class="need-chip ${chip.cls}${done?' done':''}">${display}</div>
  }).join('');
}

function handleImpactPhoto(e){
  const file=e.target.files[0]; if(!file)return;
  const r=new FileReader();
  r.onload=ev=>{
    impactB64=ev.target.result.split(',')[1];
    gv('impact-preview').src=ev.target.result;
    gv('impact-preview').style.display='block';
    gv('impact-ph-ph').style.display='none';
    gv('impact-photo-zone').classList.add('has-img');
  };
  r.readAsDataURL(file);
}

async function runImpactScan(){
  const desc=gv('impact-desc').value.trim();
  if(!impactB64&&!desc){alert('Add a photo or description first.');return;}
  gv('impact-scan-btn').disabled=true;
  gv('impact-loading').classList.add('show');
  gv('impact-result').classList.remove('show');
  impactEntry=null;

  const content=[];
  if(impactB64)content.push({type:'image',source:{type:'base64',media_type:'image
  const prompt=impactB64&&desc
    ?`Food item. Description: "${desc}". Give total macros for the whole portion
    :impactB64?'Identify this food and give total macros for the whole portion sh
    :`Total macros for: "${desc}". Use official label if branded.`;
  content.push({type:'text',text:prompt});

  try{
    const res=await fetch(PROXY,{method:'POST',headers:{'Content-Type':'applicati
      body:JSON.stringify({model:'claude-sonnet-4-20250514',max_tokens:400,
        system:'Nutrition expert. Return ONLY valid JSON, no markdown: {"name":"f
```

```
        messages:[{role:'user',content}]})});
    const data=await res.json();
    const raw=data.content.map(b=>b.text||'').join('').replace(/```json|```/g,'')
    const parsed=JSON.parse(raw);
    impactEntry={name:parsed.name,emoji:parsed.emoji||'🍽️',calories:Math.round(pa
    renderImpactResult(parsed);
  }catch(err){
    alert('Analysis failed. Try again.');console.error(err);
  }finally{
    gv('impact-scan-btn').disabled=false;
    gv('impact-loading').classList.remove('show');
  }
}

function renderImpactResult(parsed){
  const t=getTotals(),tgt=getCalTarget();
  const remP=TARGETS.p-t.p,remC=TARGETS.c-t.c,remF=TARGETS.f-t.f,remCal=tgt-t.cal
  const p=parsed.protein||0,c=parsed.carbs||0,f=parsed.fat||0,cal=parsed.calories

  gv('impact-item-name').textContent=`${parsed.emoji||'🍽️'} ${parsed.name}`;

  // Score
  let score=0;
  if(remP>5&&p>0) score+=Math.min(p/remP,1)*35;
  if(remC>5&&c>0) score+=Math.min(c/remC,1)*25;
  if(remF>5&&f>0) score+=Math.min(f/remF,1)*20;
  if(remCal>50&&cal>0) score+=Math.min(cal/remCal,1)*20;
  if(p>remP+10) score-=15;
  if(cal>remCal+100) score-=20;
  score=Math.max(0,Math.min(100,Math.round(score)));

  let scoreLabel,scoreCls;
  if(score>=75){scoreLabel='Great fit 🔥';scoreCls='score-great';}
  else if(score>=50){scoreLabel='Good fit ✓';scoreCls='score-good';}
  else if(score>=25){scoreLabel='Okay fit';scoreCls='score-ok';}
  else{scoreLabel='Poor fit';scoreCls='score-poor';}
  gv('impact-score').textContent=scoreLabel;
  gv('impact-score').className=`impact-score ${scoreCls}`;

  // Macro bars
  const bars=[
    {lbl:'Protein',cur:t.p,add:p,tgt:TARGETS.p,col:'var(--pc)'},
    {lbl:'Carbs',  cur:t.c,add:c,tgt:TARGETS.c,col:'var(--cc)'},
    {lbl:'Fat',    cur:t.f,add:f,tgt:TARGETS.f,col:'var(--fc)'},
    {lbl:'Cals',   cur:t.cal,add:cal,tgt:tgt,  col:'var(--blue2)'},
  ];
  gv('impact-bars').innerHTML=bars.map(b=>{
```

```
    const curPct=Math.min(b.cur/b.tgt*100,100);
    const addPct=Math.max(0,Math.min((b.cur+b.add)/b.tgt*100,100)-curPct);
    const after=b.lbl=='Cals'?Math.round(b.cur+b.add)+'kcal':Math.round(b.cur+b.
    const afterCol=(b.cur+b.add)>b.tgt*1.05?'var(--red)':b.col;
    return `<div class="ibar-row">
      <div class="ibar-lbl" style="color:${b.col}">${b.lbl}</div>
      <div class="ibar-track">
        <div style="position:absolute;top:0;left:0;height:100%;width:${curPct}%;b
        <div style="position:absolute;top:0;left:${curPct}%;height:100%;width:${a
      </div>
      <div class="ibar-after" style="color:${afterCol}">${after}</div>
    </div>`;
  }).join('');

  gv('impact-verdict').innerHTML=`<strong>Verdict:</strong> ${parsed.verdict||''}
  gv('impact-result').classList.add('show');
}

let _impactLock=false;
function addImpactMeal(){
  if(!impactEntry||_impactLock)return;
  _impactLock=true;
  meals.push(impactEntry);
  save(`${KEY}_meals_${todayKey()}`,meals);
  renderAll();
  closeImpactModal();
  const todayBtn=document.querySelector('.nb');
  if(todayBtn)showPage('today',todayBtn);
  setTimeout(()=>{_impactLock=false;},800);
}

// ═══════════════════════════════════════════════════════════════════════
// FEATURE: WEEKLY SUMMARY & STREAK
// ═══════════════════════════════════════════════════════════════════════
function getWeekData(){
  const days=[];
  const now=new Date();
  for(let i=6;i>=0;i--){
    const d=new Date(now);d.setDate(now.getDate()-i);
    const key=d.toISOString().slice(0,10);
    const dayMeals=load(`${KEY}_meals_${key}`,[]);
    const dayWater=parseInt(localStorage.getItem(`${KEY}_cups_${key}`)||'0');
    const dayWhoop=load(`${KEY}_whoopsnaps_${key}`,[null,null,null]);
    const t=dayMeals.reduce((a,m)=>({cal:a.cal+m.calories,p:a.p+m.protein,c:a.c+m
    const isToday=i===0;
    const hasData=dayMeals.length>0;
    days.push({key,d,t,dayWater,dayWhoop,hasData,isToday,label:d.toLocaleDateStri
```

```
  }
  return days;
}

function calcStreak(){
  let streak=0;
  const now=new Date();
  for(let i=0;i<60;i++){
    const d=new Date(now);d.setDate(now.getDate()-i);
    const key=d.toISOString().slice(0,10);
    const dayMeals=load(`${KEY}_meals_${key}`,[]);
    if(dayMeals.length>0)streak++;
    else if(i>0)break; // today allowed to be empty (in progress)
  }
  return streak;
}

function renderWeekly(){
  const el=gv('weekly-content');if(!el)return;
  const days=getWeekData();
  const streak=calcStreak();
  const tgt=getCalTarget();

  // Averages over days with data
  const logged=days.filter(d=>d.hasData);
  const avgCal=logged.length?Math.round(logged.reduce((a,d)=>a+d.t.cal,0)/logged.
  const avgP=logged.length?Math.round(logged.reduce((a,d)=>a+d.t.p,0)/logged.leng
  const daysOnTarget=logged.filter(d=>d.t.cal>=tgt*0.8&&d.t.cal<=tgt*1.15).length

  let html=`
  <div class="week-streak" onclick="openStreakModal()" style="cursor:pointer">
    <div class="streak-fire">🔥</div>
    <div class="streak-num">${streak}</div>
    <div class="streak-lbl">day streak</div>
    <div style="margin-left:auto;font-size:20px;color:var(--muted);font-weight:30
  </div>
  <div class="week-stats-row">
    <div class="wk-s"><div class="wk-v" style="color:var(--blue2)">${avgCal||'—'}
    <div class="wk-s"><div class="wk-v" style="color:var(--pc)">${avgP||'—'}g</di
    <div class="wk-s"><div class="wk-v" style="color:var(--cyan)">${daysOnTarget}
  </div>
  <div class="week-bars">`;

  const maxCal=Math.max(...days.map(d=>d.t.cal),tgt,1);
  days.forEach(d=>{
    const h=Math.max(4,Math.round((d.t.cal/maxCal)*80));
    const col=d.t.cal===0?'var(--glass2)':d.t.cal>tgt*1.1?'var(--red)':d.t.cal>=t
```

```
    const borderStyle=d.isToday?'border:2px solid var(--blue2);border-bottom:none
    html+=`<div class="wk-bar-wrap${d.isToday?' today':''}">
      <div class="wk-bar-val" style="color:${col}">${d.t.cal?Math.round(d.t.cal):
      <div class="wk-bar" style="height:${h}px;background:${col};opacity:${d.isTo
      <div class="wk-bar-lbl">${d.label}</div>
      <div class="wk-bar-dot" style="background:${d.hasData?'var(--blue2)':'var(-
    </div>`;
  });
  html+=`</div>`;

  // Per-day detail list
  html+=`<div class="week-days">`;
  [...days].reverse().forEach(d=>{
    if(!d.hasData&&!d.isToday)return;
    const lbl=d.isToday?'Today':d.d.toLocaleDateString('en-US',{weekday:'long',mo
    const recovery=d.dayWhoop[0]?.recovery;
    const recBadge=recovery!=null?`<span class="wk-badge" style="background:${rec
    html+=`<div class="week-day-row">
      <div class="wkd-lbl">${lbl}</div>
      <div class="wkd-stats">
        <span style="color:var(--blue2)">${Math.round(d.t.cal)||'—'} kcal</span>
        <span style="color:var(--pc)">${Math.round(d.t.p)||'—'}g P</span>
        <span style="color:var(--cyan)">${d.dayWater} 💧</span>
        ${recBadge}
      </div>
    </div>`;
  });
  html+=`</div>`;

  el.innerHTML=html;
}

// ═══════════════════════════════════════════════════════════════════════
// FEATURE: AUTO-SUGGEST — what to eat based on what's missing
// ═══════════════════════════════════════════════════════════════════════
async function generateSuggestions(){
  const btn=gv('suggest-btn');
  const el=gv('suggest-results');
  if(!btn||!el)return;
  btn.disabled=true;btn.textContent='Thinking…';
  el.innerHTML='<div style="color:var(--muted);font-size:12px;padding:8px 0">Gene

  const t=getTotals(),tgt=getCalTarget();
  const remCal=Math.round(tgt-t.cal);
  const remP=Math.round(TARGETS.p-t.p);
  const remC=Math.round(TARGETS.c-t.c);
  const remF=Math.round(TARGETS.f-t.f);
```

```
  const ctx=`Borna has eaten today: ${Math.round(t.cal)} kcal, ${Math.round(t.p)}
Remaining targets: ${remCal} kcal, ${remP}g protein, ${remC}g carbs, ${remF}g fat
Time of day: ${new Date().getHours()}:00. Goal: fat loss cut phase.`;

  try{
    const res=await fetch(PROXY,{method:'POST',headers:{'Content-Type':'applicati
      body:JSON.stringify({model:'claude-sonnet-4-20250514',max_tokens:600,
        system:'Return ONLY valid JSON, no markdown: {"suggestions":[{"name":"foo
        messages:[{role:'user',content:ctx}]})});
    const data=await res.json();
    const raw=data.content.map(b=>b.text||'').join('').replace(/```json|```/g,'')
    const parsed=JSON.parse(raw);

    el.innerHTML='';
    for(const s of parsed.suggestions){
      const div=document.createElement('div');
      div.className='sugg-item';
      div.innerHTML=`
        <div class="sugg-top">
          <span class="sugg-emoji">${s.emoji}</span>
          <div class="sugg-name">${s.name}</div>
          <button class="sugg-add" title="Add to log">+</button>
        </div>
        <div class="sugg-reason">${s.reason}</div>
        <div class="sugg-macros">
          <span style="color:var(--blue2)">${s.calories} kcal</span>
          <span style="color:var(--pc)">${s.protein}g P</span>
          <span style="color:var(--cc)">${s.carbs}g C</span>
          <span style="color:var(--fc)">${s.fat}g F</span>
        </div>`;
      div.querySelector('.sugg-add').addEventListener('click',()=>{
        let addLock=false;
        if(addLock)return;addLock=true;
        meals.push({name:s.name,emoji:s.emoji,calories:s.calories,protein:s.prote
        save(`${KEY}_meals_${todayKey()}`,meals);
        renderAll();
        div.querySelector('.sugg-add').textContent='✓';
        div.querySelector('.sugg-add').disabled=true;
      });
      el.appendChild(div);
    }
  }catch(e){
    el.innerHTML='<div style="color:var(--muted);font-size:12px;padding:8px 0">Su
  }finally{
    btn.disabled=false;btn.textContent='↻ Refresh';
  }
```

```
}

// Hook renderAll to also call new renderers
const _origRenderAll=renderAll;
window.renderAll=function(){
  _origRenderAll();
  renderNutrients();
  renderWeekly();
  renderStreakMini();
};
// Call it once now to init
renderNutrients();
renderWeekly();
renderStreakMini();

// ═══════════════════════════════════════════════════════════════════════════
// FEATURE: STREAK MINI-CARD (today page)
// ═══════════════════════════════════════════════════════════════════════════
function renderStreakMini(){
  const el=gv('smc-num');
  if(!el)return;
  el.textContent=calcStreak();
}

// ═══════════════════════════════════════════════════════════════════════════
// FEATURE: STREAK MODAL
// ═══════════════════════════════════════════════════════════════════════════
let _sfRaf=null;

function openStreakModal(){
  const modal=gv('streak-modal');
  if(!modal)return;
  const streak=calcStreak();

  // Hero count-up
  const numEl=gv('sm-num');
  if(numEl){
    let n=0;const dur=600;const start=performance.now();
    function tick(now){
      const p=Math.min((now-start)/dur,1);
      numEl.textContent=Math.round((1-Math.pow(1-p,3))*streak);
      if(p<1)requestAnimationFrame(tick);
    }
    requestAnimationFrame(tick);
  }

  // Started date
```

```
const startedEl=gv('sm-started');
if(startedEl&&streak>0){
  const d=new Date();d.setDate(d.getDate()-(streak-1));
  startedEl.textContent=d.toLocaleDateString('en-US',{month:'short',day:'numeri
} else if(startedEl) startedEl.textContent='—';

// Rank
const rankEl=gv('sm-rank');
if(rankEl) rankEl.textContent=streak>=30?'Elite':streak>=14?'Strong':streak>=7?

// All-time max
const maxEl=gv('sm-max');
if(maxEl){
  let max=0,cur=0;
  for(let i=0;i<180;i++){
    const d=new Date();d.setDate(d.getDate()-i);
    const m=load(`${KEY}_meals_${d.toISOString().slice(0,10)}`)||[];
    if(m.length>0){cur++;if(cur>max)max=cur;}else cur=0;
  }
  maxEl.textContent=`${Math.max(max,streak)}d`;
}

// Week dots Mon-Sun
const dotsEl=gv('sm-dots');
if(dotsEl){
  const dayLabels=['M','T','W','T','F','S','S'];
  const today=new Date();const dow=today.getDay();
  const mondayOffset=dow===0?-6:1-dow;
  let html='';
  for(let i=0;i<7;i++){
    const d=new Date(today);d.setDate(today.getDate()+mondayOffset+i);
    const k=d.toISOString().slice(0,10);
    const isFuture=d>today&&k!==todayKey();
    const m=load(`${KEY}_meals_${k}`)||[];
    const lit=m.length>0&&!isFuture;
    html+=`<div class="sm-dot"><div class="sm-dot-circle ${lit?'lit':'dim'}">${
  }
  dotsEl.innerHTML=html;
}

// Milestone bar
const milestones=[7,14,30,60,90,180,365,730];
const nextM=milestones.find(m=>m>streak)||365;
const prevM=[...milestones].reverse().find(m=>m<=streak)||0;
const pct=nextM>prevM?Math.min(((streak-prevM)/(nextM-prevM))*100,100):100;
const mbar=gv('sm-mbar');if(mbar)setTimeout(()=>mbar.style.width=pct+'%',80);
const mlabel=gv('sm-mlabel');
```

```
  if(mlabel)mlabel.textContent=streak>=nextM?`🏆 ${nextM}-day milestone reached!`

  // Callout
  const msgs=[[0,'Just Getting Started 💪','Every legend starts at day 1. Log you
  const ctitle=gv('sm-callout-title'),cbody=gv('sm-callout-body');
  if(ctitle&&cbody){const msg=[...msgs].reverse().find(([d])=>streak>=d)||msgs[0]

  _startStreakFlame(streak);
  modal.classList.add('active');
}

function closeStreakModal(){
  const modal=gv('streak-modal');
  if(modal)modal.classList.remove('active');
  if(_sfRaf){cancelAnimationFrame(_sfRaf);_sfRaf=null;}
}

function _newFlameParticle(W,H){
  return{x:W/2+(Math.random()-0.5)*W*0.5,y:H*0.85+Math.random()*H*0.1,vx:(Math.ra
}

function _startStreakFlame(streak){
  const canvas=gv('streak-flame-canvas');if(!canvas)return;
  const dpr=window.devicePixelRatio||1;
  const W=260,H=220;
  canvas.width=W*dpr;canvas.height=H*dpr;
  canvas.style.width=W+'px';canvas.style.height=H+'px';
  const ctx=canvas.getContext('2d');ctx.scale(dpr,dpr);
  const t=Math.min(streak/90,1);
  const r1=Math.round(255*(1-t)+181*t),g1=Math.round(120*(1-t)+40*t),b1=Math.roun
  const r2=Math.round(255*(1-t)+90*t),g2=Math.round(60*(1-t)+0*t),b2=Math.round(0
  let particles=Array.from({length:60},()=>_newFlameParticle(W,H));
  function frame(){
    ctx.clearRect(0,0,W,H);
    if(particles.length<80)particles.push(_newFlameParticle(W,H));
    particles.forEach(p=>{
      p.x+=p.vx+Math.sin(p.phase+Date.now()*0.002)*0.4;
      p.y+=p.vy;p.life-=p.decay;p.phase+=0.05;
      if(p.life<=0)return;
      const cr=Math.round(r1+(r2-r1)*(1-p.life)),cg=Math.round(g1+(g2-g1)*(1-p.li
      ctx.beginPath();ctx.arc(p.x,p.y,p.r*p.life,0,Math.PI*2);
      ctx.fillStyle=`rgba(${cr},${cg},${cb},${p.life*0.85})`;ctx.fill();
    });
    particles=particles.filter(p=>p.life>0);
    _sfRaf=requestAnimationFrame(frame);
  }
  if(_sfRaf)cancelAnimationFrame(_sfRaf);
```

```
  frame();
}

// ═══════════════════════════════════════════════════════════════════════════
// FEATURE: STYKU STATS BAR AUTO-SCROLL (ping-pong)
// ═══════════════════════════════════════════════════════════════════════════
let _sbRaf=null,_sbDir=1,_sbPause=0;

function _sbFrame(){
  const row=document.querySelector('.sb-row');
  if(!row){_sbRaf=null;return;}
  const maxScroll=row.scrollWidth-row.clientWidth;
  if(maxScroll<=0){_sbRaf=requestAnimationFrame(_sbFrame);return;}
  if(_sbPause>0){_sbPause--;_sbRaf=requestAnimationFrame(_sbFrame);return;}
  row.scrollLeft+=_sbDir*0.6;
  if(row.scrollLeft>=maxScroll-1){row.scrollLeft=maxScroll;_sbDir=-1;_sbPause=120
  else if(row.scrollLeft<=1){row.scrollLeft=0;_sbDir=1;_sbPause=120;}
  _sbRaf=requestAnimationFrame(_sbFrame);
}

function initStykuScroll(){
  const row=document.querySelector('.sb-row');if(!row)return;
  row.addEventListener('touchstart',()=>{if(_sbRaf){cancelAnimationFrame(_sbRaf);
  row.addEventListener('touchend',()=>{setTimeout(()=>{_sbPause=60;_sbRaf=request
  setTimeout(()=>{_sbRaf=requestAnimationFrame(_sbFrame);},2000);
}

initStykuScroll();
```